ChatGPT

Scooby-Doo Face Detection & Recognition Roadmap

This roadmap outlines a milestone-driven plan to build a **classic CV/ML face detection and character recognition system** for Scooby-Doo images. The solution uses a sliding-window detector with HOG-like features and a Linear SVM, without any deep learning or pre-trained models. We emphasize implementing everything from scratch (with allowed libraries like NumPy, OpenCV, skimage, sklearn, matplotlib) and following the required folder structure, naming conventions (e.g. all source code under a `cod/` folder), dataset format, and `.npy` output specifications. Each milestone defines clear deliverables (files, outputs, figures), a "definition of done", and verification steps. We also include conceptual explanations, pseudocode snippets, debugging visualizations, common pitfalls, and best practices for reproducibility. The goal is to **incrementally build** a working baseline and then iteratively improve it, helping the student deeply understand each component while assembling the project correctly from the ground up.

1. Locked Deliverables & Definition of Done (per Milestone)

Below is a summary of expected deliverables for each milestone, including files (code, model, output) and visualizations. Achieving these and passing the specified checks define the "done" criteria for that stage:

- **Milestone M0 – Project Setup & Baseline**:
- **Project structure in place**: Create the required directories and files. All source code files reside in `cod/` (per project spec). Use the provided skeleton file names and interfaces (e.g. `FacialDetector.py`, `Parameters.py`, `RunProject.py`, `Visualize.py`) to ensure compatibility.
- **Dummy output generation**: Implement a minimal *baseline detector* that produces dummy outputs in the required format (e.g. an empty or random set of detections) for all test images. Save example `.npy` files (`detections_all_faces.npy`, `scores_all_faces.npy`, `file_names_all_faces.npy`) in the proper solution folder hierarchy. This tests that the evaluation script can run end-to-end with your output format.

- **Definition of Done**: Directory structure matches specification; code runs without errors; dummy `.npy` outputs load correctly; the evaluation script (if available) runs on the dummy output (likely yielding 0% AP or random results) without crashing.

- **Milestone M1 – Data Exploration & Preparation**:

- **Data loading and sanity checks**: Code to read training images and annotations (e.g. from `antrenare/` subfolders and annotation text files). Print dataset stats (number of images, faces per class, image resolutions, etc.).
- **Visualization outputs**: Save a **histogram of ground-truth face bounding box sizes** (width/height distribution) to `debug/face_size_hist.png` to understand scale variability. Save a sample

image with ground-truth boxes overlaid (`debug/gt_boxes_sample.png`) to verify annotation parsing is correct.

- **Definition of Done**: All training and validation annotations are parsed into data structures (e.g. lists of bounding boxes per image and their class); no errors in reading files; visual inspection of the saved debug images confirms correctness (histogram looks reasonable; sample image shows boxes aligning with faces).

- **Milestone M2 – Feature Extraction & Sample Prep**:

- **Positive and negative sample generation**: Extract **face patches** from training images (positives) and **non-face patches** (negatives) at the required window size (e.g. 36×36). Save these cropped examples to disk or memory for debugging.
- **HOG feature computation**: Compute HOG descriptors for all training patches. Cache the feature matrices as `.npy` (e.g. `pos_features.npy`, `neg_features.npy`) for reuse. Each positive feature is a vector of length as specified (e.g. 900 if using 36×36 with 6×6 cells [1]).
- **Debug visuals**: Save a **grid image of positive vs. negative patches** (`debug/patches_grid.png`) – e.g. a montage with a few example face crops and a few non-face crops side by side – to visually confirm the samples. Save a **HOG visualization** of a prototype face (`debug/hog_visualization.png`), showing the gradient orientation histograms overlayed on the image (using skimage's `hog(..., visualize=True)`). This should reveal a rough face shape if computed correctly.

- **Definition of Done**: The number of extracted positives equals the count of annotated faces; the number of negatives meets the set budget (e.g. 10,000 negatives as required). Each HOG feature vector has the expected length and type. Unit-test style checks pass (e.g. an assert that HOG feature length == expected, printed "[OK] HOG dimension = …"). The patch grid image shows clear difference: face crops contain characters' faces, negatives look like background or other parts (no faces). No face appears in negative samples (verified via annotation overlap checks).

- **Milestone M3 – Initial Face Detector (Train & Single-Scale Test)**:

- **SVM training**: Train a linear SVM classifier for **face vs. non-face** using the HOG features (e.g. using `sklearn.svm.LinearSVC`). Save the trained model coefficients (weights and bias) to disk (if needed) or keep in memory. Possibly output the learned **"template" HOG weights as an image** (`debug/hog_face_template.png`) for curiosity – when reshaped to the HOG cell grid, strong gradients might outline a face.
- **Single-scale detection**: Implement a basic sliding-window detector at one scale (the base window size). For each validation image, slide a 36×36 window (or whatever `dim_window` is) across the image (e.g. in steps of one HOG cell or a few pixels) and classify each window using the SVM. Collect all detections with scores above a threshold (initially 0). **Non-Maximum Suppression (NMS)** is applied to these detections to filter overlapping ones. Save **detection result images** for a couple of validation samples: one image showing **all raw detections** (many overlapping rectangles), and another showing **detections after NMS** (cleaned, one rectangle per face) – e.g. `debug/detections_before_nms.png` and `debug/detections_after_nms.png`.
- **Output files**: Generate detection outputs on the **validation set** in the format required for Task 1: `.npy` files for all faces – `detections_all_faces.npy`, `scores_all_faces.npy`,

`file_names_all_faces.npy`. (These should contain every detected bounding box, its confidence score, and the corresponding image filename for each detection, likely sorted by score.) Compute the **Precision-Recall (PR) curve** and Average Precision (AP) on the validation set by comparing against `validare/task1_gt_validare.txt`. Save the PR plot as `debug/pr_curve_initial.png` with AP labeled.

- **Definition of Done**: SVM training completes with a reasonable model (e.g. prints training accuracy or loss; `[OK] SVM trained on X positives + Y negatives`). The detector runs through all validation images in a reasonable time. The PR curve is generated – it might be modest (e.g. AP ~0.2–0.3 if only single-scale) [2], but **not zero**, indicating some faces are detected. The NMS output images show that faces are detected with one box each (and not dozens of overlapping boxes). The `.npy` output files conform to the exact naming and format specs (correct filenames, array shapes, data types). The evaluation script (if run on validation) produces a meaningful AP and shows no format errors.

- **Milestone M4 – Multi-scale & Hard Negative Mining (Detector Refinement)**:

- **Multi-scale pyramid**: Extend the detector to handle **multiple scales** of the image. Implement an image pyramid (e.g. scale factor ~0.8–0.9 per step) and run the sliding window at each scale. This greatly improves recall for faces larger or smaller than the base window [2]. Update the detection code to collect boxes from all scales, then apply NMS globally. Save a **pyramid visualization** (`debug/image_pyramid.png`) showing one image rescaled to a few levels (stacked or side-by-side) to illustrate your scaling.
- **Hard Negative Mining**: Using the current detector, run it on a set of **negative images** (images with no Scooby-Doo characters, if provided, or on the training images themselves but looking at regions that **do not contain any annotated face**). Collect high-scoring false positives (e.g. any detection with score above a low threshold, or the top N per image) [3]. Verify none of these overlap a true face (use IoU < 0.3 criterion to ensure they are truly background). Add these "hard" negatives to the training pool and re-train the SVM (or fine-tune it by continuing training) [3]. This should force the classifier to fix its mistakes and reduce false alarms. For efficiency, you might skip NMS in this mining step to gather even overlapping false positives (since they're all negatives) [4]. Limit how many hard negatives you add (e.g. at most as many as the initial negatives, to avoid imbalance).
- **Re-training and results**: After retraining with hard negatives, run the multi-scale detector again on validation. Save a **comparison PR curve** (`debug/pr_curve_after_hard_neg.png`) overlayed with the initial curve to show improvement (e.g. higher precision at given recall). Log the **AP before vs. after** (expect a modest boost of a few points [5]). Also record any change in detector behavior (e.g. "[OK] Hard negatives added: 500, AP improved from 0.30 to 0.35").
- **Deliverables**: Updated SVM model (final weights). Possibly new debug images: e.g. `debug/hard_negatives_examples.png` (a grid of some false positive patches that were added as negatives, to inspect what the detector was confusing as faces). Final tuned detection outputs on validation (new `.npy` files and PR plot). If required, prepare the **final test set outputs** for Task 1 as well (using the refined detector on `testare/` images, producing `detections_all_faces.npy` etc. in the submission folder).

- **Definition of Done**: The multi-scale detector achieves significantly higher recall than single-scale (e.g. detects faces of varied sizes that were missed before – visible by more true positives in PR curve, possibly AP closer to $\geq$0.5 depending on data). False positives are reduced after hard-negative mining (higher precision at low recall – fewer spurious detections). No new bugs introduced

(e.g. ensure the pyramid loop and coordinate scaling are correct – verify by checking a detection's scale and comparing to original image size). The final detector is **"good enough"** when adding more hard negatives yields little to no AP improvement (stop criterion could be ≤1% AP gain or a set maximum iterations of mining). All outputs still conform to required formats and naming.

- **Milestone M5 – Character Recognition (Multi-class Task 2)**:

- **Multi-class classification setup**: Prepare training data for recognizing *which* character is in the face. Using the annotated class labels, compile a dataset of **face HOG features labeled by character** (Daphne, Fred, Shaggy, Velma). (This can reuse the same face patches from training used for detection – now just with class labels instead of "face"). If necessary, balance the classes (if one has far fewer examples, you might consider data augmentation like horizontal flips, though optional).
- **Train character classifier**: Use a linear model (e.g. a LinearSVC or LogisticRegression) for multi-class classification. This can be a single multi-class SVM (one-vs-all internally) or four one-vs-rest SVMs. Input features are the face HOG vectors; output is one of the four names. Tune regularization if needed to handle multi-class data. Save the model (or the 4 sets of weights).
- **Integrated detection+recognition**: For each validation image, run the **face detector** (from M4) to get face bounding boxes. For each detected face, extract its HOG feature (or even the raw patch) and predict the character using the classifier. (Ensure to use the same HOG parameters as training.) Each detection now gets a class label and a confidence score. Produce the required Task 2 output files: for each character, a `.npy` file of detections belonging to that class, with corresponding scores and filenames. (This essentially splits your detections by predicted class – e.g. `detections_daphne.npy`, `scores_daphne.npy`, `file_names_daphne.npy`, and similarly for fred, shaggy, velma.)
- **Evaluation & metrics**: Using the validation ground truth for Task 2 (`task2_<name>_gt_validare.txt` for each class), evaluate the recognition performance. Compute the **confusion matrix** for character classification: rows = actual class, cols = predicted class. Each face should count towards either the correct class or a misclassification. Also compute per-class Precision, Recall, and F1-score, and overall accuracy. Save the **confusion matrix plot** as `debug/confusion_matrix.png` (with percentages or counts annotated). Additionally, plot **per-class PR curves** if required (or at least note the AP for each class detector if treating as detection tasks per class). Also prepare some **example results images**: e.g. `debug/example_detections_task2.png` showing a few test images with bounding boxes labeled with the predicted character name (color-code by class).
- **Deliverables**: Final code in `cod/` that integrates detection and recognition. All final `.npy` files for Task 1 and Task 2 (for submission, likely placed in the `evaluare/fisiere_solutie/<StudentID_Name>/...` folder as required). A short **report (PDF)** that includes the key figures and quantitative results (see checklist below).
- **Definition of Done**: The system detects faces and correctly identifies a good portion of them. For example, the confusion matrix should show strong diagonal values (most faces classified correctly) and low off-diagonals. Each class has decent recall (no character is completely missed). If some classes perform worse, the issues are analyzed (perhaps due to fewer training samples or very similar looking characters in certain images). The final code produces the outputs in the exact format the evaluator expects (double-check file names, array shapes, and that detection boxes align with image coordinates). At this point, the full pipeline is ready for the official test run on unseen data.

2. Milestone Objectives, Inputs/Outputs, Effort, and Stop Criteria

M0 – Project Setup & Baseline
- **Objective:** Initialize the project structure and verify that the pipeline and evaluation can run with a trivial baseline.
- **Inputs:** Provided dataset directories (train/antrenare, validation/validare, etc.), skeleton code files, project specs (naming conventions).
- **Outputs:** `cod/` source code folder with initial code files; dummy `.npy` output files for detection (all faces).
- **Effort (est.):** ~4 hours.
- **Stop Criteria:** Directory and file naming match spec; code runs end-to-end (no crashes); evaluation script runs on dummy output (yielding expected poor metrics).

M1 – Data Exploration & Preparation
- **Objective:** Read and understand the dataset; prepare for feature extraction by verifying annotations and data characteristics.
- **Inputs:** Annotation text files (`*_annotations.txt`), training images per class, validation ground truth files.
- **Outputs:** In-memory data structures for images & face boxes; debug visuals (GT box overlay, face size histogram).
- **Effort:** ~1 day (including writing parsing code and plotting).
- **Stop Criteria:** Able to iterate through all training images and annotations without error; distribution of face sizes is reasonable (no extreme outliers or indicates if multi-scale is needed); sample visualization confirms parsing correctness.

M2 – Feature Extraction & Sample Prep
- **Objective:** Implement HOG feature extraction for positive (face) and negative (non-face) samples; assemble the training feature dataset.
- **Inputs:** Loaded images and annotations from M1.
- **Outputs:** Cropped face patches (possibly saved or just used on the fly); negative patches; HOG feature arrays for each (pos_features, neg_features); debug images (patch grids, HOG vis).
- **Effort:** ~1–2 days (coding HOG extraction, testing on small examples, tuning cell size).
- **Stop Criteria:** Feature extraction functions return correct sizes (e.g. for a 36×36 patch with cell=6, feature length 900 as expected [1]); no faces in negatives (verified via IoU checks or visual scan); ready to train SVM with these features.

M3 – Initial Face Detector (Train & Single-Scale Test)
- **Objective:** Train a baseline face detector and test it on validation images at the base scale.
- **Inputs:** HOG feature matrices and labels from M2.
- **Outputs:** Trained Linear SVM model (coefficients); detection results on validation (boxes, scores); PR curve and AP on validation; detection visualization images.
- **Effort:** ~1 day (training is quick; main effort in implementing sliding window and NMS, and verifying outputs).
- **Stop Criteria:** Detector finds at least some faces (precision/recall > 0 with a few true positives); AP is low but non-zero (~0.1–0.3 baseline) [2]; code can generate the required .npy output format correctly. We move on when we have a working end-to-end pipeline, even if performance is far from perfect.

M4 – Multi-scale & Hard Negative Mining
- **Objective:** Improve the detector's recall and precision by scanning multiple scales and by retraining with hard negatives.
- **Inputs:** Trained model from M3; original training images (for mining negatives); possibly separate negative-only images.
- **Outputs:** Enhanced detector model; new detection results on validation (likely higher AP); comparison PR curves; possibly updated threshold for final output.
- **Effort:** ~1–2 days (implementing pyramid + NMS, running detection, mining negatives, retraining, repeating if necessary).
- **Stop Criteria:** Achieve a "good enough" detector – e.g. AP on validation meets project expectation (if given) or improves significantly from M3. Diminishing returns on adding more hard negatives or adjusting scales indicate it's time to proceed. The system should now detect most faces (high recall) with manageable false positives.

M5 – Character Recognition (Task 2)
- **Objective:** Extend the face detector into a face recognizer distinguishing the main characters.
- **Inputs:** Per-class face data (from training annotations); refined face detector from M4.
- **Outputs:** Multi-class classifier model; final detection + recognition outputs on validation (confusion matrix, per-class metrics); code integrated to produce Task 2 `.npy` files for test.
- **Effort:** ~1 day (prepare data, train classifier, integrate pipeline, evaluate).
- **Stop Criteria:** End-to-end system correctly identifies a substantial portion of faces. All required output files for both tasks are produced in correct format. At this stage, the project is ready for final testing and report writing.

(Note: The time estimates assume familiarity with tools and are approximate. Always leave buffer for debugging. The key is to timebox each milestone – if a step is "good enough" (e.g. performance plateaus or basic functionality is achieved), proceed rather than perfecting endlessly.)

3. Milestone M0: Project Setup & Baseline

Concept & Explanation: Milestone 0 is about laying the groundwork and ensuring we have a runnable framework. It's crucial to organize the project exactly as required. We create the directory structure and blank or template files for code and outputs. By running a trivial baseline (even one that outputs no detections or random detections), we test that the evaluation pipeline can read our results. This prevents format mistakes later and confirms our environment (Python, libraries) is set up correctly.

Implement Steps (Baseline):
- **Create required folders:** According to the spec, set up directories like `cod/` (for all source code), `debug/` (for saving debug images), and the expected output folder structure for results. For example, if submission requires outputs under a specific path (e.g. `evaluare/fisiere_solutie/<ID_Name>/task1/`), ensure we mirror that in our project for testing. Put an empty `__init__.py` in `cod/` if needed to treat it as a package. - **Copy/setup starter code:** Place the provided skeleton files (if given) into `cod/`. Typical files are: - `Parameters.py` – for global settings (we will fill in defaults here, like `dim_window`, `dim_hog_cell`, thresholds, etc). - `FacialDetector.py` – likely a class with methods to train, detect, etc. We'll be implementing functions here in later milestones. - `RunProject.py` – a script to execute the training and detection pipeline in order. - `Visualize.py` – perhaps utilities for plotting images, etc.

Ensure these filenames remain exactly as provided (the evaluation script may import them by name). - **Baseline dummy detector:** Before any real model, implement a placeholder in `RunProject.py` (or equivalent) that generates an output file for each required `.npy`. For example, create empty numpy arrays for detections and scores:

```
import numpy as np
# Suppose we have N_test images (this can be read from test directory)
file_names = []
detections = np.zeros((0,4), dtype=np.int32)  # no detections
scores = np.array([], dtype=np.float32)
np.save('evaluare/fisiere_solutie/331_Alexe_Bogdan/task1/
detections_all_faces.npy', detections)
np.save('evaluare/fisiere_solutie/331_Alexe_Bogdan/task1/scores_all_faces.npy',
scores)
np.save('evaluare/fisiere_solutie/331_Alexe_Bogdan/task1/
file_names_all_faces.npy', np.array(file_names))
```

(Replace the path with the appropriate one for your environment; the key is to produce the three files.) This example writes an empty detection set. Alternatively, one could output one random rectangle per image to simulate a random detector [6]. The content doesn't matter as long as format is correct. - **Terminal output checks:** Print confirmation messages as the baseline runs. E.g., "Loaded X test images – writing dummy detections [OK]". This ensures the code is actually executing as expected. - **Run evaluation (if available):** Use the provided `evalueaza_solutie.py` on the fake output. It should not crash due to file or format issues. It will likely report 0% recall and AP (if no detections) or some number if random. That's fine – it confirms the pipeline wiring. If the evaluation script isn't available to students, manually verify format by loading the npy files and comparing with ground truth (they should be empty or nonsense at this point by design).

Verify & Debug:
- Check that `RunProject.py` finishes without errors. If any file path issues arise (common on Windows vs Unix paths), fix them now. Print the current working directory to ensure relative paths are correct. - Verify the dummy `.npy` files exist and have the right shape/dtype. For example, load `detections_all_faces.npy` and confirm it's shape (0,4) for an empty array of 4 columns (representing `[x, y, w, h]` or `[x1,y1,x2,y2]` as required by spec – clarify which format is needed and stick to it). - Confirm the code structure: All source code is under `cod/` and we will run our project via something like `python cod/RunProject.py`. If the evaluation expects a certain entry-point or class, ensure we will provide that (for instance, if they expect a class `FacialDetector` with a `run()` method, plan to implement that accordingly). - If using version control, commit this initial state (tag it as baseline). This helps in case we break something later – we know at least this dummy version ran without issues.

Common Pitfalls (M0) and Fixes:
- *Symptom:* **Evaluation script crashes or cannot find files.**
Cause: Folder or file naming is wrong (e.g. placed outputs in the wrong directory or named file slightly differently than expected).
Diagnosis: Read error messages or evaluation code to see which file/path is missing. Double-check against the project spec example paths.

Fix: Rename or move files to exactly match expected names. For instance, if the spec says output files should be in `task1/` subfolder, ensure that's where you put them, not in the project root. Keep consistent casing and underscores. Running a quick `os.listdir()` in code to print out the directories can help verify paths.

- *Symptom:* **Import errors or module not found** when running `RunProject.py`.
  Cause: The `cod` folder might not be recognized as a module, or PYTHONPATH not set.
  Diagnosis: If using relative imports in the code, you might need an `__init__.py` or adjust sys.path.
  Fix: Add an `__init__.py` in `cod/`. When running, do `python -m cod.RunProject` from the parent directory, or adjust the working directory so that imports like `from cod.FacialDetector import FacialDetector` work. Ensure the structure is correct so that when the project is packaged, the evaluator can run it easily.

- *Symptom:* **Output .npy file contains wrong data type or shape.** (e.g., evaluator expected int coordinates but got floats, etc.)
  Cause: Numpy default types or our generation code mismatch the spec.
  Diagnosis: Check the spec or evaluation code for how it reads the .npy. If coordinates are supposed to be ints (pixel indices), ensure `dtype=np.int` for coordinates. If using floats for scores, that's fine but ensure it's `float32` or `float64` consistently.
  Fix: Cast or specify dtype when saving arrays. Also ensure the shape convention: the detector output likely expects each detection as [x_min, y_min, x_max, y_max] or [x, y, width, height]. Clarify and stick to it. For baseline, it might not matter, but for the real system it will. We will confirm and use the correct format in later milestones.

Once M0 is done, we have a scaffold to build upon, with confidence that our environment and pipeline scaffold are correct. We can then tackle actual data and algorithms.

4. Milestone M1: Data Exploration and Preparation

Concept & Explanation: In this phase, we focus on **understanding the dataset** and preparing the data for feature extraction. We need to read the annotated images and possibly split data if needed (training vs validation). A thorough data exploration helps avoid mistakes later (e.g., knowing typical face sizes informs our HOG window size and pyramid scale range). We'll implement functions to load images and annotations, and verify them by visualization.

Key Steps:
- **Parse annotations:** The training annotations are given in text files like `daphne_annotations.txt` (likely listing image file names and face bounding boxes). We write a parser that reads each line, extracts image name and coordinates, and stores them. We might structure it as a dict mapping image -> list of boxes (with class info). Since the training images are sorted by character (in separate folders), the annotation file name itself indicates the class. We can use that: e.g., for each line in `daphne_annotations.txt`, label those boxes with class "daphne". Repeat for others. Alternatively, each class's images might only contain that class's face(s), but to be safe we'll rely on annotations for exact coordinates. - **Load images:** Using OpenCV (`cv2.imread`) or matplotlib (`plt.imread`), load each training image (likely color images). Convert to grayscale if we plan to use HOG on grayscale (common

practice, since color usually isn't used for HOG). Store images in memory or keep file paths and load on demand to save RAM. Also load validation images and their ground truth if needed for any initial checks. - **Explore dataset statistics:** Compute how many faces in total, how many per class, and the distribution of face bounding box sizes. We iterate over all annotated boxes: - Compute width = x_max - x_min, height = y_max - y_min for each. Gather these in a list. Plot a histogram of widths and heights (or of the max dimension or area). This gives insight into what base window size might be suitable. For example, if most faces are around 40×40 pixels, a 36×36 window might be appropriate. If there's a wide range, multi-scale is definitely needed. Save this histogram plot (as `debug/gt_face_size_hist.png`). Ensure to label axes (pixel count) and maybe mark the chosen window size. - **Verify one example visually:** Pick one image (maybe the first in the Daphne set). Draw the ground truth bounding box (e.g., using cv2.rectangle or matplotlib patches) on the image. If multiple faces in one image, draw all relevant boxes (for that class or all classes if you combine). Save this as `debug/gt_example.png`. This simple sanity check often catches if our coordinates were interpreted incorrectly (e.g., swapped or off by one). If the box aligns perfectly over the face, we know our parsing and coordinate system are correct. - **Plan negative data sources:** Check if we have dedicated "negative" images (ones with no characters). The folder structure didn't list a separate negatives directory. Possibly we will use portions of training images that don't contain any face as negatives, or additional provided images in "fake_test/" as negatives. Plan: we can treat any region not overlapping an annotation in any image as a potential negative. Also, if any provided "background" images exist, note their paths for later use. We don't actually extract negatives yet (that's M2), but we formulate how we will choose them (randomly or grid sampling). - **Set up Parameter config:** In `Parameters.py`, start filling known values: e.g., `dim_window` (we might set 36 based on historical use [7]), `dim_hog_cell` (6 as default [7]), etc. Also define `overlap_threshold = 0.3` (for NMS IoU, as given [8]) and placeholders for any other hyper-params. This central config will help ensure consistency across code.

Mini Exercise (Data Parsing): After writing the annotation parser, test it on a small snippet:

```
# Example snippet to test annotation parsing
annotations = load_annotations('antrenare/daphne_annotations.txt')
print(list(annotations.items())[:1])  # print first image's entry
```

Manually verify the printed coordinates against the text file or image to ensure parsing logic (like splitting line by spaces or commas) is correct.

Verification & Debugging:
- Ensure **image file paths** are correct. If the annotation gives just a filename, you may need to prepend the folder path (e.g., "antrenare/daphne/<filename>.jpg"). A common bug is wrong relative paths. To catch this, try opening one image path and handle exceptions. - Check that the number of images in annotation files matches actual image files present. Print counts. If any mismatch, double-check file naming (maybe extensions or cases). - Inspect the **histogram of sizes**. If it shows, for example, a peak around 30–50 pixels, then our chosen `dim_window=36` is reasonable. If faces vary from 20 up to 100 pixels, we know multi-scale pyramid must span at least ~0.5x to ~3x scaling. Make a note of this range for M4 when implementing the pyramid. If the histogram has a tail of very large faces, that might be outliers or maybe group shots (less likely in cartoons). - The **ground truth overlay image** should clearly show the box tightly around the face. If it's offset or wrong size, fix the parsing: - Check if coordinates in annotation are perhaps given as center + width/height instead of corners. (The structure naming `*_gt_validare.txt` suggests ground

truth boxes; perhaps each line format is "filename x y w h" or "filename x_min y_min x_max y_max". We must confirm from context or instructions.) By drawing and seeing if it matches the face, we deduce the format. If it's shifted, maybe it was width/height instead of max coords. - Once confirmed, adopt a consistent box format throughout (prefer [x_min, y_min, x_max, y_max] internally for ease of IoU calculation). - Print out a couple of annotation entries in console in a readable way and manually verify with the actual images if possible.

Common Pitfalls (M1) and Fixes:
- *Symptom:* **Annotation box drawn in wrong location or size** (e.g., not covering the face).
Cause: Misinterpreted annotation format (swapped coordinates order, wrong indexing). For instance, assuming (x_min,y_min,x_max,y_max) but file might be (x,y,w,h).
Diagnosis: Compare one annotation line to the image. If the drawn box's width equals the supposed x_max coordinate, you likely treated width as x_max.
Fix: Adjust the parser accordingly. If format is (x, y, w, h), convert to (x_min, y_min, x+w-1, y+h-1) for consistency. Add comments in code clarifying the expected format, e.g., "Annotations given as top-left + width,height, converting to corners."

- *Symptom:* **Face size histogram shows impossibly large values or is empty.**
  Cause: Could be a parsing error (like reading strings as ints failed) or not iterating correctly. Or maybe using the wrong unit (if images were resized elsewhere?).
  Diagnosis: Print min, max face width/height explicitly. If you see something like max width = 2000 pixels in a 800×600 image, something's off.
  Fix: Re-evaluate the coordinate extraction. Possibly the annotation file units are not pixels (less likely) or there's a bug reading lines (like concatenating two numbers). Ensure you strip newline and split properly. If empty, make sure you actually read the file (file path correctness).

- *Symptom:* **Mismatch between annotation count and actual images.**
  Cause: Perhaps some images have multiple faces (so appear in multiple annotation files for each class). Or an image file is missing.
  Diagnosis: Cross-reference by image name. If the same image (e.g., a group photo) is listed in multiple class annotation files (each listing the face of one character), you might double-count or override entries.
  Fix: Decide on a unified structure: one image can have multiple faces of different classes. So you might want a dictionary `all_annotations[image] = list of (box, class)` for all classes. Implement merging: loop over each class file and append to the image's list. This way, if image X has Daphne and Fred, it ends up with two entries. This ensures when we use that image for training detector (which doesn't care who, just that there are faces) we know it has two positive windows, and also for task 2 we retain class labels. If any image listed in annotations isn't found in the filesystem, flag it – maybe the dataset expects you to skip or there's a typo.

At the end of M1, we should have a solid understanding of our data. We'll know how to crop positive examples and where to get negatives. We proceed knowing our ground truth parsing is correct, which is crucial for both training (using correct face crops) and for evaluating our detector later (matching IoUs correctly).

5. Milestone M2: Feature Extraction and Sample Preparation

Concepts: This milestone brings in core computer vision concepts: **Histogram of Oriented Gradients (HOG)** for feature extraction, and the preparation of positive and negative samples for training. We will also implement an important utility: **Intersection over Union (IoU)**, which will be used to filter negatives and later for NMS and evaluation. By the end of M2, we'll have a large feature matrix for faces and one for non-faces, ready to feed into the classifier.

- **HOG Feature Descriptor:** HOG, introduced by Dalal and Triggs (2005), is a representation of the local shape (gradients) of an image patch [9] [10]. We divide the window (e.g. 36×36) into small cells (e.g. 6×6 pixels). For each cell, we compute a histogram of gradient orientations (typically 9 bins for 0–180°). We group cells into blocks (usually 2×2 cells) for normalization, which gives a block descriptor (in Dalal-Triggs, each block yields a 36-length vector). All such blocks in the window form the HOG feature vector for that window [1]. The result is a feature vector that's somewhat illumination-insensitive and captures edge patterns like eyes, nose, chin outline for a face. We can use `skimage.feature.hog` to compute this, rather than coding from scratch, since the focus is not to reinvent HOG but to use it. The parameters (cell size, block size, etc.) must be consistent between training and detection.
- **IoU (Intersection over Union):** IoU is a metric to quantify overlap between two boxes [11]. It's defined as: $$ \text{IoU}(B_\text{pred}, B_\text{gt}) = \frac{\text{Area}(B_\text{pred} \cap B_\text{gt})}{\text{Area}(B_\text{pred} \cup B_\text{gt})}.$$ It ranges from 0 (no overlap) to 1 (perfect overlap) [12] [11]. We'll implement a function `compute_iou(box1, box2)` that returns this ratio. IoU is vital for:
- Filtering negatives: we consider a sample "negative" only if IoU with *any* ground truth face in that image is below a small threshold (to ensure it doesn't accidentally include part of a face).
- Non-Max Suppression: to decide when two detections are "too close" and one should be suppressed (often if IoU > 0.3, we consider them the same object) [8].
- Evaluating detection: a predicted box is counted as correct if IoU $\geq$ 0.5 with a ground truth [13] (common threshold in detection benchmarks).

Implementation Steps:

1. **Crop Positive Patches:** Iterate through each training image and its face annotations. Extract the pixel region for each face. If the face boxes are not already the target window size (`dim_window`), we have to decide: do we **resize** the face patch to `dim_window` × `dim_window`? In many pipelines (like Dalal-Triggs human detection), all positive examples are normalized to the same size for training [14]. We will do the same (e.g., resize each face to 36×36) so that our HOG features are consistent. Use `cv2.resize` with interpolation. (Alternatively, if all faces are roughly the same size already or if we want to include scale variation in training, we could avoid resizing – but given classic approaches, we'll normalize sizes).
2. If faces vary a lot in aspect ratio or scale, consider just resizing to the base size; minor distortion is okay for our task. Ensure conversion to grayscale if not already.
3. Store each face patch (maybe append to a list `pos_patches` for debugging or saving if needed) and then compute HOG for it directly to get features.

4. **Sample Initial Negative Patches:** We need a set of image patches that do **not** contain faces. A straightforward approach:

5. For each training image (or dedicated negative images if any), we can take a number of random crops of size `dim_window`. However, random crops might sometimes accidentally cut through a face if not careful. So:
6. Use the IoU check: when picking a random crop in an image that has faces, compute IoU with all ground truth face boxes in that image. If IoU < 0.1 (some small number) for all faces, then we accept it as a negative patch. This ensures it's well away from any actual face.
7. We can also sample negatives from images known to have no faces (if such are provided, e.g. background scenes).
8. Alternatively, do a systematic scan: slide a window through the image and collect those windows with IoU=0 for all faces. But that could yield an enormous number of negatives; better to randomly pick a manageable number.
9. Aim for, say, at least as many negatives as positives (the Parameters default was 10,000 negatives for 6,713 positives [15]). We can start with ~2x or ~3x the number of positives to ensure robust training (since the background class is very broad).
10. **Mini-toy approach:** Initially, just pick e.g. 100 random negative patches to test the process, before scaling up to thousands.

11. Store negative patches in list `neg_patches`.

12. **Compute HOG Features:** Now use `skimage.feature.hog` on all these patches:

13. Determine `pixels_per_cell = dim_hog_cell` (from Parameters, say 6) and `cells_per_block = (2,2)` as standard [16].
14. `feature_vector=True` to get a flat array per patch (or False to get the block array which we can flatten manually – either works).
15. For speed, you might vectorize or parallelize this if needed, but initially just loop.
16. Collect all positive feature vectors in an array `X_pos` of shape (N_pos, D) and negative in `X_neg` of shape (N_neg, D), where D is descriptor length (should be (cells_x -1)*(cells_y -1)*36; e.g. 900 for 36×36 with 6px cells [1]).
17. Print out `X_pos.shape, X_neg.shape` and `D` to verify dimensions. For example, if we have ~500 faces and 1000 negs for a quick test, we expect (500,900), (1000,900).

18. Also, confirm a single HOG output manually: take one known patch (maybe a simple synthetic image or a face) and compute hog. Check `hog_image` from skimage's visualize to see that the shape is (cells_y-1, cells_x-1, 36) if not flattened [10]. Flattening that should give D.

19. **Prepare training data structures:** We will combine these features and create labels for training the SVM:

20. `X_train = np.vstack([X_pos, X_neg])`
21. `y_train = np.hstack([ np.ones(len(X_pos)),  np.zeros(len(X_neg)) ])` (Using 1 for face, 0 for non-face). Or use -1 for non-face if preferred by SVM convention. Sklearn's LinearSVC typically expects labels 0/1 or -1/1 – check docs; we can use 1 and -1 to clearly indicate positive vs negative.

22. We might not train the SVM here yet (that's M3), but it's good to have the data ready. If memory is an issue, we can skip combining now and combine on-the-fly in training.

23. Save these arrays to disk using `np.save` (especially if feature computation took long, caching is good). E.g., save `pos_HOG.npy` and `neg_HOG.npy` in a cache folder or `cod/` so that if we tweak SVM, we don't redo HOG each time.

24. **Unit tests for IoU and other functions:** Before relying on IoU for real filtering:

25. Test `compute_iou` with known boxes. Example:

```
boxA = [0,0,10,10]
boxB = [0,0,10,10]
assert compute_iou(boxA, boxB) == 1.0, "[ERROR] IoU of identical boxes
should be 1"
boxC = [10,10,20,20]
assert compute_iou(boxA, boxC) == 0.0, "[ERROR] IoU of non-overlapping
boxes should be 0"
boxD = [5,5,15,15]
iouAD = compute_iou(boxA, boxD)
# Intersection area 25 (5x5 square from (5,5) to (10,10)), Union area 175,
IoU = 25/175 = 0.142...
assert abs(iouAD - 0.142) < 1e-3, "[ERROR] IoU calculation seems off"
print("[OK] IoU function passed basic tests")
```

26. This gives confidence that IoU works. If any fail, debug the logic (common mistake: using width+height instead of right coordinate, etc.). We can cite the pseudocode from V7 labs as reference: we do `inter_w = min(x2_A, x2_B) - max(x1_A, x1_B)` etc [17] [18].

27. Also test that our HOG extraction makes sense: e.g., compute HOG on a blank image vs an image with a simple edge and see that the feature values differ accordingly. Skimage's HOG might be difficult to interpret directly, so relying on visualize output is fine. At least verify the length of feature vector matches expectation.

28. **Debug Visualizations:** We generate a few images to ensure our samples are correct:

29. **Positive vs Negative grid (`debug/patches_grid.png`):** Take e.g. 5 random face patches and 5 random negative patches. Resize them to a common size for display (they should already be 36×36 if we normalized). Create a 2-row grid where top row is faces, bottom row is negatives. This visual comparison helps ensure negatives indeed look like background or non-face patterns (if a negative patch accidentally had a face or part of it, we would spot it and realize our sampling is wrong).
30. **HOG visualization (`debug/hog_vis_example.png`):** Take one representative face patch (perhaps centered, frontal face). Use `hog(..., visualize=True)` to get the `hog_image`. Rescale it for visibility and save. The image should show gradient lines. We expect to see an outline of the face in the hog visualization (e.g., an indication of where the gradients are strongest – around chin, hair, etc.). This confirms that our HOG is capturing features. If it looks all blank or random, something's off (maybe the image was flat or parameters wrong).

31. **Intensity check:** (Optional) Print min/max of a HOG feature vector. HOG features are often normalized, so values should be in a reasonable range (0 to 1). If we see huge values or NaNs, likely an issue (like not converting image to float [0,1] range – skimage expects images either uint8 0-255 or float 0-1; if we gave float 0-255, that could mess up gradients scale). Fix by ensuring input image type is correct (`skimage.hog` will internally call `float(image)` if not already, but ensure grayscale dtype is float or uint8).

Verification & Criteria:
- We have `N_pos` positive feature vectors and `N_neg` negatives. Check `N_pos` matches the total faces count from M1. `N_neg` should be roughly what we aimed for (if far fewer, perhaps the IoU criterion was too strict and we rejected too many – we might then loosen it or take more random tries). - The feature dimensionality `D` should be consistent for all; print a sample vector shape. If using `feature_vector=True`, `X_pos[0]` will be length D. If we accidentally used `feature_vector=False`, we'd have a 3D array (cells_y-1, cells_x-1, 36). In that case, fix by flattening: e.g. `feat.ravel()` or use `hog(..., feature_vector=True)`. - The `[OK]` print from IoU tests should appear. Also consider a quick test: IoU to filter negatives – if a training image has a face, test one negative crop near that face to see if IoU function correctly identifies overlap. For example, if a face is at (50,50,86,86) and we take a window at (40,40,76,76), IoU would be significant (overlap of 36×36). Ensure our IoU logic catches that (should be ~0.7 IoU). - Also verify **speed considerations**: computing thousands of HOGs can be slow. Since this is a one-time preprocessing, it's okay if it takes a minute or two, but if it's extremely slow, consider optimizing: - Use grayscale images to avoid computing gradients for 3 channels. - Perhaps use OpenCV's HOGDescriptor as an alternative if needed (OpenCV might be faster in C++). - If still in development, you might limit number of negatives just to get through, then increase later. Or use multi-threading (skimage HOG might release GIL? If not, consider chunking features and using joblib or similar). - Once done, print a summary: e.g., "Generated 500 positive and 1500 negative samples. HOG feature length = 900. [OK]" This confirmation will be helpful in logs.

Common Pitfalls (M2) and Fixes:
- *Symptom:* **HOG feature length or values are incorrect.**
Cause: Parameter mismatch or forgetting to resize images. For example, if a face patch isn't exactly `dim_window`, skimage.hog will produce a different sized descriptor.
Diagnosis: If one patch yields length 900 and another yields 884, likely their sizes differ.
Fix: Always resize input patch to `dim_window`. Or, if you deliberately use variable sizes, ensure you specify `feature_vector=False` and then resize the HOG output via interpolation which is complex. Simpler: stick to fixed patch size. Also ensure `pixels_per_cell` divides the window size evenly (36/6=6 cells per side which minus block overlaps yields 5 blocks, works; if it didn't divide evenly, skimage might pad or return a smaller last cell). So choose cell size factors properly. The default given in Parameters (6 px, with 36 window) is consistent [1].

- *Symptom:* **Negative samples accidentally contain faces (or parts of them).**
  Cause: IoU threshold too low or bug in IoU. If IoU calc is wrong, it might return 0 for overlapping areas, allowing bad negatives.
  Diagnosis: Spot-check some negative patches near faces. Or visualize a negative that IoU should have caught.
  Fix: Correct the IoU implementation (test with known scenarios as above). Possibly increase the exclusion buffer – use IoU < 0.2 or check even the center of the window not falling inside a face box.

It might also help to discard negatives that are very uniform (no gradients) since they don't teach the classifier much – but that's a minor point.

- *Symptom:* **Memory issues with feature arrays.**
  Cause: Extremely large number of samples or not releasing image arrays. 10k negatives * 900 features ~ 9 million values, which is fine (72 MB if float64). But if using float64 instead of float32, double memory.
  Diagnosis: Monitor memory. If it swaps, maybe too many negatives at once.
  Fix: Use `dtype=np.float32` for feature arrays to halve memory (sklearn SVM is fine with float32). Or generate and train in batches rather than storing all at once (not needed unless extremes). Also free image arrays if not used (in Python, ensure no lingering references so GC clears them). Usually not a big issue at this scale.

- *Symptom:* **HOG visualization is blank or not interpretable.**
  Cause: Possibly the face patch contrast is low or the visualization scaling.
  Diagnosis: Try a patch with distinct edges (maybe a cartoon face has strong edges). If still nothing, perhaps not converting color to gray (skimage.hog on a color image will internally convert or flatten all channels which might dilute results).
  Fix: Ensure grayscale input. Also stretch contrast of hog_image for display if needed (the `hog()` function returns a visualization that may need normalization to 0-255 for saving). Use `cv2.normalize` or simply rescale by multiplication. In any case, this is a debug aid – if it's not clear, it's not critical, but it usually should show some gradient structure.

By completing M2, we now have the "learning" data prepared: `X_train` and `y_train` with rich HOG features for faces and non-faces. We also have utilities like IoU that will be reused. We likely also fixed our window size decisions and know the hog parameters. Next, we tackle training the classifier and building the actual detector.

6. Milestone M3: Train Initial Face Detector and Single-Scale Detection

Concepts: Now we move to the **machine learning** part: training a classifier (Linear SVM) and using it to detect faces by scanning images. We'll also implement **Non-Maximum Suppression (NMS)** to clean up multiple detections. This milestone yields the first functional face detector. Key concepts here include the SVM decision function, sliding window scanning, and how detection results are evaluated (Precision-Recall).

- **Linear SVM classifier:** A Support Vector Machine finds a hyperplane that separates positive (face) and negative (non-face) examples with maximum margin. We use a linear kernel so the model is basically a weight vector `w` and bias `b` the same length as our feature. Classification is `score = w·x + b`. We want `score > 0` for faces, <0 for non-faces. We likely use `sklearn.svm.LinearSVC` (which optimizes a linear SVM) or `SVC(kernel='linear')`. We must choose a regularization parameter `C` (inverse of lambda). A smaller C means more regularization (larger margin, possibly more misses on training data). We can start with default or a moderate C (like 0.01) and adjust if needed (if we see lots of misclassifications on training vs val).
- **Sliding Window Detection:** With a trained classifier, detecting faces in an image means computing the classifier score on *every possible window* of the image of size 36×36 (at various scales in general).

For this milestone, we start with single-scale (i.e., assume faces are roughly 36×36 in the image already). We slide the window typically with some step (e.g. 4 pixels or one cell, which is 6 px in our case, for efficiency). At each position, extract the patch, compute its HOG, and get SVM score. This is expensive if done naively for every pixel; using HOG we can optimize by computing the HOG of the whole image once and then sliding a *window in HOG space*. For simplicity initially, we might just brute-force: iterate x from 0 to width-36, y from 0 to height-36, step 6, compute hog for each window (skimage has an option to compute an entire image's HOG and then sub-window it, but it's complex to implement from scratch). Given moderate image sizes, brute force might be okay to start (we can optimize later if needed).

- **Non-Maximum Suppression (NMS):** Once we have many scored windows, we need to filter out redundant detections. NMS algorithm [19] :
- Take all detection windows above a score threshold (e.g. >0 initially, or we might take the top N if too many).
- Sort them by score descending (highest confidence first).
- Create an empty list `keep`.
- For each detection in sorted order, if it does not have high IoU (>0.3) with any detection already in `keep`, then keep it (append to `keep` list). Otherwise, discard it as a duplicate of a higher-score detection of presumably the same face.
- Return the `keep` list as final detections. This ensures we only have one detection per face. We use our IoU function here with threshold 0.3 (Parameter.overlap) [8] , meaning if a new box overlaps more than 30%, we consider it the same object and ignore the new one.
- **Scoring and Ranking:** We will output all kept detections with their scores. For evaluation (Precision-Recall), the detections are typically sorted by score and then matched to ground truth. Our `.npy` output likely should be sorted by score already (to simplify evaluator's job, though it may not be strictly necessary – but usually one does). So we will sort final detections by confidence before saving. Also, include *all* detections above some min score (even low ones) because in PR, false positives with lower scores still contribute at low precision end. The evaluation script likely will handle different score thresholds automatically to build the PR curve [20] . So don't be overly aggressive with a high cutoff; use a minimal threshold (like 0 or slightly below if needed to get higher recall).
- **Precision-Recall (PR) and Average Precision:** Precision = TP/(TP+FP), Recall = TP/(TP+FN). As we adjust the detection score threshold, we get different precision/recall. The PR curve plots Precision vs Recall for all operating points. AP is roughly the area under this curve (and in practice computed by summing precision differences over recall increments). We will use the evaluator or compute ourselves by matching detections to ground truth with IoU>=0.5 as a correct match [13] . At this stage, AP might be low. But we expect at least some improvement over random. This is a measure of detector quality. We'll output the PR graph to visually see how many false positives vs recall we have.

Implementation Steps:

1. **Train the SVM:** Using the `X_train, y_train` from M2. For reproducibility, set `random_state` in LinearSVC. Example:

```
from sklearn.svm import LinearSVC
clf = LinearSVC(C=0.01, loss='hinge', max_iter=10000, random_state=42)
clf.fit(X_train, y_train)
```

```
print(f"[OK] SVM trained with {len(X_train)} samples, coef shape =
{clf.coef_.shape}")
```

Check that `coef_.shape` is (1, D) for binary classifier. The bias is `clf.intercept_`. Optionally evaluate training accuracy:
`train_pred = clf.predict(X_train); np.mean(train_pred == y_train)`. It might be high (we expect >90% on training if not too regularized, but that's okay).

2. If training is slow or fails to converge, we might reduce sample count or increase max_iter. LinearSVC by default uses a squared hinge loss which is fine. We could also try `sklearn.linear_model.SGDClassifier` for incremental training (especially if doing hard negative mining later), but LinearSVC is okay for moderate data sizes.
3. If there is class imbalance (negatives >> positives), ensure the SVM doesn't get biased to always predict negative. We might set `class_weight='balanced'` or manually adjust, but often if you have enough negatives, a well-chosen C will handle it. Monitor if training accuracy on negatives is extremely high at expense of missing positives – that could mean we overfit negatives.
4. Once trained, retrieve `w` and `b`: `w = clf.coef_.reshape(-1); b = clf.intercept_[0]`. We can later use these directly for computing scores as `score = w @ feature + b`.

5. **Implement Detector on Single Scale:** For each validation image:

6. Compute its HOG feature map. To optimize: Instead of computing HOG for every sub-window separately (which would redo gradient calc a million times), we can compute the full image's HOG once. `skimage.hog(image, pixels_per_cell=(6,6), cells_per_block=(2,2), feature_vector=False)` will give an array of HOG blocks. If image size is W×H, the HOG output shape will be (H_cell-1, W_cell-1, 36) where H_cell = H/6, W_cell = W/6 (assuming divisible). Each cell is 6 px, block stride is presumably cell stride, so effectively each position in that HOG output corresponds to a block of 2×2 cells in the image.
7. Then, sliding a 36×36 window corresponds to sliding a 6×6 cell window in the HOG grid (since 36/6 = 6 cells). But careful: a 36×36 patch has 5×5 = 25 blocks (if cells_per_block=(2,2) and we slide one cell step at a time, a 36×36 window yields 5 blocks across and down) [1]. Actually, using the `feature_vector=False` output, one could extract a sub-array of shape (5,5,36) for each window position. But implementing that indexing is a bit complex but doable:
    - If `hog_image` is the HOG features for the whole image (size ~ (H/6 -1, W/6 -1, 36)), then for a window at cell position (i,j) (i in [0, H_cell-6], j in [0, W_cell-6] presumably), the block features for that window are `hog_image[i:i+5, j:j+5, :]`, which should be a 5×5×36 block grid that corresponds to that window. Flatten that to get a 900-D vector.
8. As a simpler approach (with some performance cost, but clarity), we can slide in the image domain:

```
step = 6  # slide one cell at a time
win = param.dim_window  # 36
for y in range(0, image_height - win + 1, step):
    for x in range(0, image_width - win + 1, step):
        patch = image[y:y+win, x:x+win]
        feat = hog(patch, pixels_per_cell=(6,6), cells_per_block=(2,2),
feature_vector=True)
```

```
        score = clf.decision_function([feat])[0]
        if score > threshold:
            detections.append([x, y, x+win, y+win])
            scores.append(score)
            file_names.append(img_name)
```

This brute-force approach is easier to implement correctly. If it's too slow on large images, we can try to optimize after verifying correctness (maybe by using the hog feature map method).

9. The `threshold` initially can be 0 (meaning we take any window classified as face by the SVM). Possibly even slightly negative if we want high recall (the PR evaluation will sort them anyway). We could store *all* window scores and later filter in NMS stage; but storing millions of windows is not feasible. Instead, apply a reasonable cutoff to reduce candidates. 0 is a good start, or perhaps the smallest positive score or top N per image like 1000 if that's easier.
10. Collect all detections across all validation images. We might actually do NMS per image (which is typical, since NMS is usually done within each image's detections). The evaluation expects the final list of detections across images though. So:
    - For each image, run detection, apply NMS to get final boxes for that image. Add those to global list.
    - Alternatively, combine all detections then do NMS – but that would suppress across images too, which we don't want. So do it image by image.
    - However, when saving results, we output them all together with corresponding filenames.
11. **Implement NMS:** Given a list of boxes and scores for one image:

```
keep_indices = []
idxs = np.argsort(scores)[::-1]  # sort by score desc
for i in idxs:
    box_i = boxes[i]
    suppress = False
    for j in keep_indices:
        if compute_iou(box_i, boxes[j]) > 0.3:
            suppress = True
            break
    if not suppress:
        keep_indices.append(i)
final_boxes = [boxes[k] for k in keep_indices]
final_scores = [scores[k] for k in keep_indices]
```

This is O(n^2), but n (the number of detections above threshold in an image) might be on the order of maybe a few hundred after thresholding, which is fine. If it were thousands, one can optimize by sorting and using coordinate-based suppression, but not needed for moderate numbers.

12. After NMS, store `final_boxes, final_scores, final_file_names` into the global result lists.

13. **Create output numpy files:** Now we have global lists of detections (each detection is [x_min,y_min,x_max,y_max]), scores, and file_names (image name repeated for each detection). Convert them to numpy arrays and `np.save`:

```
np.save('task1/detections_all_faces.npy', np.array(detections))
np.save('task1/scores_all_faces.npy', np.array(scores))
np.save('task1/file_names_all_faces.npy', np.array(file_names))
```

Make sure that `file_names` is an array of dtype `'<U...` (string). The evaluation script will likely load these and iterate over them. The image names should probably include extension and match exactly the names in the ground truth file. (E.g., "image1.jpg"). If ground truth doesn't have extension, see if needed, but likely it does.

14. Sorting by score: The evaluator might sort itself, but to be safe we might sort the *global* detections by score as well (especially if the evaluation computes AP by taking top k detections overall). Many academic eval scripts assume the list is already sorted by confidence [19] . So after collecting all image results, do a global sort by score descending. However, ensure that the matching of file_names and detections remains aligned after sorting (i.e., sort indices and apply to all three lists/arrays).

15. If concerned, we can read the `validare/task1_gt_validare.txt`: it likely expects no particular order, just as long as all detections are present to compute per-image metrics. But AP as a single number typically requires sorting by score globally. We'll assume sorting is expected or at least harmless.

16. **Evaluate on Validation:** Use the provided `evalueaza_solutie.py` with `--task 1` on the `task1` output files. It should output precision, recall, AP. If we don't have that script usage, implement a quick evaluation:

17. For each ground-truth face in validation, find if there's a detection box with IoU≥0.5. That's a true positive. If multiple detections match the same face, only one counts (others are false positives – NMS should have removed them though).
18. For each detection, decide if it's TP or FP by matching to an unmatched GT. Compute precision and recall at various score thresholds. This is non-trivial to implement perfectly, but since we have ground truth, we can at least compute overall recall (#TP / #GT) and precision at, say, the threshold we used. But a full PR needs doing this at different cuts. Probably easier: rely on evaluation code or approximate AP by sorting our detections by score and walking down:
    - Sort detections by score desc. Iterate and at each detection, mark TP if it matches an unclaimed GT, else FP. Compute precision=TP/i, recall=TP/total_GT at each i. Then plot those.

19. Log a summary: e.g., "Validation: {TP}/{GT} faces detected, recall = 0.6, precision = 0.2 at threshold=0. Score threshold can be tuned." And "AP ≈ 0.X".

20. **Visualization and Debug:**

21. Plot the PR curve using matplotlib: use the data from evaluation (if provided it might directly give AP). Save `debug/pr_curve_initial.png`.
22. On a few images, overlay the detection results **after NMS**: draw a rectangle and maybe put the score above it. Save those images (we already did in M3 deliverables perhaps).

23. Also, it's educational to visualize **before NMS**: e.g., one image with a dozen overlapping rectangles on one face. Save that to illustrate why NMS is needed.
24. Check if any obvious faces were missed (false negatives). If yes, examine those images to see why:
    - Maybe face was too large or too small for this single scale – expected, we only handle ~36 px faces now. So large faces might only have a corner detected etc.
    - Maybe face was slightly rotated or profile – HOG + linear SVM might struggle if not enough similar training data. If some systematic pose issues, note them (can't do much except get more data or maybe augment with mirrored faces).
25. Check false positives that remain after NMS: is there a pattern? Perhaps some textured background or parts of characters' bodies triggering the face classifier. This will motivate Hard Negative Mining in M4.

Common Pitfalls (M3) and Fixes:

- *Symptom:* **Very few (or zero) detections found on validation (low recall).**

Cause: Could be a too high threshold or a classifier issue. If threshold=0 and still nothing, possibly the SVM is underfitting (predicting almost everything as negative). Maybe C is too high (over-regularized) or we have too many easy negatives.

Diagnosis: Check some window scores manually. For an image with a face, what score does the face window get? If it's negative, then the classifier didn't actually learn the face well. Print the range of scores for an image containing a known face.

Fix: Possibly increase C to make the SVM fit training data more (if it was underfit). Or check training data – were positives correct and sufficient? Another cause can be mis-scaled HOG features (if not normalized similarly as training, but since training and detection use same HOG function, that should be fine). Ensure no bug in applying classifier (e.g., using `clf.predict` might threshold at 0 by default, better use `decision_function` for raw score). Use `decision_function` for continuous scores. If using `predict`, it gives binary labels only, which loses ranking info. So always accumulate `decision_function` results.

- *Symptom:* **Detector finds faces but with multiple overlapping boxes per face.**

  Cause: NMS not working as intended. Maybe IoU threshold too low or boxes coordinates inconsistent.

  Diagnosis: If the overlapping boxes don't get suppressed, either our IoU function might be computing low IoU when it's actually high (e.g., if we use width/height as x_max mistakenly).

  Fix: Re-test IoU with two nearly identical boxes to ensure it returns high IoU. Possibly the coordinate convention: if our detections are [x_min,y_min,x_max,y_max], ensure IoU function interprets that correctly. If they were [x,y,w,h], IoU code would be wrong. Standardize to corners. Also, check threshold: if we set 0.3, perhaps that was too low to remove duplicates that have IoU 0.5. Typically for face, could even use 0.5. But our spec said 0.3 [8], which is aggressive (removes even fairly loosely overlapping). Actually, lower threshold = stricter removal (even slight overlaps cause suppression) – maybe they chose 0.3 to be safe to not remove distinct faces that are somewhat close. We can stick to 0.3. If we see double-box on one face, maybe those two had IoU < 0.3 (meaning they were quite offset, possibly one covering eyes region, one covering mouth – could happen if part of face also looked like a face). Hard negative mining will address those partial detections.

- *Symptom:* **Many false positives (low precision).**

  Cause: No surprise at this stage; our classifier might not be super tight. Also without multi-scale, it might be detecting face-like patterns in odd places.

Diagnosis: Look at top false positives (highest score ones that are wrong). Are they e.g. the Mystery Machine van's circular headlights mistaken as eyes, etc.? The classifier is still naive.
Fix: This will be tackled in M4 (hard negatives). In the short term, one could raise the detection threshold to reduce FP (at cost of recall). For instance, if precision is very low, maybe set threshold to 0.5 to output fewer high-confidence detections. But since AP considers the ranking, we don't actually need to cut them out; just ensure their scores are lower than any TP if possible. For now, accept that initial precision might be low.

- *Symptom:* **SVM training takes too long or memory error.**
  Cause: Too many training samples or very high dimensional features.
  Diagnosis: If LinearSVC is slow, maybe our feature dimension is large (but 900 is fine). If we had, say, used a larger window or smaller cell (3x3 cell would quadruple feature length).
  Fix: If dimension is indeed huge, consider adjusting HOG parameters (but 900 is small). If samples are like >50k, LinearSVC might struggle. Possibly use `Dual=False` in LinearSVC if N>>D, or use SGDClassifier. But our scale should be okay (e.g., 6k pos + 10k neg = 16k, which is fine). Also ensure we didn't accidentally generate negatives in an infinite loop or something.

At this point, we have a working detector that we can run on any image to get face locations (for roughly one scale). The performance might be suboptimal, but building it end-to-end is a major milestone. We can now identify shortcomings (faces missed due to scale, false positives from confusing backgrounds) and address them next.

7. Milestone M4: Multi-Scale Detection and Hard Negative Mining (Refinement)

Concepts: This milestone focuses on **improving the detector**. Two main improvements: scanning over multiple scales (to handle faces of different sizes), and **Hard Negative Mining** to fine-tune the classifier on challenging non-face examples it currently gets wrong [3]. We'll also possibly tune some parameters like the sliding step or SVM threshold to balance precision/recall.

- **Multi-Scale Pyramid:** Real-world images have faces of varying sizes. We simulate a larger face by scaling down the image and detecting at the base window size on the scaled image. Conversely, smaller faces by scaling up the image (or equivalently, scaling down the window). Typically, create an image pyramid: repeatedly downsample the image (e.g. by 0.9 or 0.8) until the image is smaller than the window. At each scale, run the detector as in M3. Alternatively, we can also upscale if needed for smaller faces (but upscaling too much gets blurry; usually we downscale only, because we can detect smaller faces in the original or downscaled images?). Actually, to detect *smaller* faces than the base 36px, we need to **upscale** the image or, equivalently, run the detector on an image scaled up. But scaling up an image that was, say, originally small might not add info. Usually one picks a base size as the smallest face to detect and then downscale image to catch larger ones. If some faces in data are smaller than 36, perhaps they expect us to miss them or maybe images are high-res enough. We should find out from the face size histogram: if many faces <36, we might consider upscaling. However, upscaling beyond original resolution doesn't create new information, so not much point.
- So likely approach: set `scale_factor = 0.9` and repeatedly multiply image by that until min(image_dim) < 36. This covers faces larger than 36 (they'll appear in some scale of the pyramid at ~36).

- For faces smaller than 36 in original, those might remain hard (maybe the dataset doesn't have very small faces, or if it does, they expect lower recall there).
- Implementation: We can re-use the single-scale detector on each level. Optimize by computing hog at each scale.
- We must adjust coordinates: if we downscale by factor, the detections found correspond to the scaled image coordinates. We need to scale them back up to original image coords when saving. Keep track of the scale factor for each level.
- Also note: smaller scale (image downsampled) means we detect *bigger* faces (because a face that was 100px might appear as ~50px in a half-sized image and get detected). Conversely, a face of 20px in original would never be detected because even at original scale our window is 36. If we upscaled 2x, that face becomes 40px and maybe detectable. If small faces are crucial, an alternative is to train a smaller window detector or just accept missing them. Given this is a cartoon project, maybe faces aren't extremely small relative to image.

- Multi-scale will drastically improve recall (with some cost in more false positives) [21] .

- **Hard Negative Mining (HNM):** After initial training, we saw some false positives. HNM procedure [3] :

- Take a set of images that **should not contain faces** (commonly, the training images that are not meant for faces or a separate negative dataset). We likely use either:
    - The same training images (which do contain faces in parts, but we skip the face regions).
    - The validation images if allowed (but better not, we usually don't retrain on validation ground truth – unless explicitly allowed as part of project).
    - Any extra negative images (like background scenes, perhaps the "fake_test" or others provided).
- Run the current detector (multi-scale) on these images with a low threshold (e.g. score > -1 or so, to get a lot of candidates).
- Collect detection windows that **do not correspond to any real face** (we ensure that by checking IoU with any ground truth in that image if it's a training image with some faces – or if it's a known no-face image, then everything it finds is a false positive).
- Among these detections, sort by score (confidence). Take the top N (maybe top 500 or 1000) "hard negatives" – these are patches the classifier is most confused by (it gave them high face-likeness scores). We don't want to take tens of thousands because many will be overlapping or very similar patches, which adds redundant info and slows training. Often adding a number comparable to initial negatives is good.
- Extract HOG features for these hard patches (we can directly use the detection pipeline's computed features if we were clever, but easier: just crop them from original image and compute hog anew).
- Add these to the negative training set and assign label 0/-1.
- Re-train the SVM with the expanded training set. Usually we don't remove the old negatives; keep them plus new ones (or possibly replace some easier ones if memory is an issue, but better keep).
- This typically improves precision because the SVM will adjust to carve out a boundary that sends those hard examples to the negative side.
- Optionally, this can be iterated: after retraining, you can do another round and find even new hard negatives. But diminishing returns – often one iteration yields most benefit (~5-10% precision boost) [22] .
- Note: There's a risk of chasing a few very tricky false positives and potentially hurting generalization (the SVM might overfit to them, especially a linear one has limited capacity but also could get

skewed). We watch if recall on validation faces drops after adding HNs. If it does, maybe we added something that caused the classifier to also reject some true faces (maybe parts of faces looked like those negatives).

- Implementation detail: perhaps skip NMS when collecting negatives (as UT advice suggests [4]) because if the detector fires multiple overlapping windows on one region (like a textured area), those are all hard negatives and we might want to feed several of them to the SVM to really teach it that entire region is not a face. However, adding many nearly-identical patches can also overweight that region. A compromise: allow some overlap, but you could enforce IoU < 0.2 between chosen hard negatives to ensure diversity. Simpler: limit one per distinct area by using NMS with high threshold (like 0.5) so you don't take 10 from same location.
- Ensure not to include any actual face: e.g., if using training images that have faces, you must exclude detections that overlap a ground truth face even slightly. Otherwise, we'd be telling the SVM that an actual face is negative – very bad (it will likely then make w mis-classify faces).

- Hard negatives can include things like characters' bodies, or parts of faces (eyes only region might have hog similar to a small face), etc. The SVM will adjust weights to reduce those scores, ideally without losing the full face detection.

- **Parameter Tuning:** This might also be a point to tune sliding window step (maybe use smaller step to not miss faces between steps – UT found step=3 px improved AP [23], though at cost of runtime). For now, step=cell size is usually fine.

- SVM threshold: after retraining, we might adjust the threshold for final output to balance precision/recall. But since AP evaluation doesn't require a fixed threshold, we can output all and let scoring handle it. However, for a nice operating point (e.g., if we want a single detection output per image as final, then threshold matters – but in this project, likely AP is the measure, so we focus on that).
- Multi-scale range and scale factor: If PR shows recall still drops beyond certain size, maybe extend scales (e.g., go down to 0.5 scale if needed or upsample 1.5× for smaller faces). But watch out for runtime.
- It's okay to allow a bit more false positives at low scores to ensure high recall; AP will handle it as long as our ranking is good.

Implementation Steps (M4):

1. **Multi-Scale Implementation:** Modify detection code:

```
scales = []
scale = 1.0
min_size = min(image_width, image_height)
while min_size * scale >= dim_window:
    scales.append(scale)
    scale *= 0.9
```

(This goes down until the scaled image is smaller than window, covering large faces. If we want to handle smaller faces than window, we could do a couple of upscales: e.g., also do scale=1.2, 1.5 if image resolution allows – be cautious.)

2. For each `scale` in scales:
    - Resize image to that scale (use cv2.resize with interpolation=cv2.INTER_LINEAR for downscaling, maybe INTER_CUBIC for upscaling).
    - Run the sliding window detector (the one we had) on this scaled image.
    - When adding a detection, scale its coordinates back: `x_orig = x/scale, y_orig = y/scale, x_max_orig = (x+win)/scale, y_max_orig = (y+win)/scale`. Because if we found a face at (x,y) in the scaled image, in original it covers more pixels.
    - Append with appropriate score and filename.
3. After processing all scales of an image, apply NMS to the combined list of detections from all scales for that image. (Alternatively, you can do NMS per scale then NMS cross-scale, but it should be equivalent to doing one NMS on all).

4. This yields final boxes for that image.

5. **Efficiency considerations:** computing HOG at each scale from scratch could be slow if we have many scales. If images are large and scale factor 0.9, number of scales ~ 10-15 maybe. For each, we compute hog and slide. It's multiply slower than single scale. If this is too slow, one trick: compute the HOG pyramid (OpenCV's HOGDescriptor can do that internally). But given moderate dataset size, it might be okay.

6. If needed, restrict to a reasonable scale range (e.g., if hardly any face is > 2× base or < base, perhaps no need to scale beyond 2× difference).

7. After multi-scale, we expect recall to improve dramatically: faces that were say 72px will now be caught at scale0.5 (downsampled to 36). Test on validation: did we detect faces that were missed earlier due to size?

8. **Apply Hard Negative Mining:** Decide what images to mine on. Good choices:

9. The training set images themselves (they contain faces, but we ignore those regions). Many background patches can be taken from them – but our initial negative set likely already had random ones. However, now we target those that fooled our classifier.
10. The validation set could be used (since we have GT for them, we know where faces are, and any detection outside those is a false positive). However, using validation to retrain blurs the line – it may be allowed since they gave GT, but it's probably meant for evaluation only. It's safer to only use training images for training, including HNM.
11. If they provided a `fake_test` or some negative images (maybe scenes with monsters? Actually any Scooby scene has at least the gang or villain usually, not sure if pure background images exist).
12. We can also re-use the initial negatives set images (if any provided like SUN database in UT assignment).
13. For each selected image, run the *current detector* (the one from M3 or from multi-scale? Ideally use multi-scale as well, to find any size false positives).
14. Use a low threshold: even slightly negative scores might be worth adding if they are among top mistakes. Perhaps collect any detection with score > -0.5 or so, but practically, focus on those above 0 (since those are ones classifier currently thinks could be face).
15. Check IoU with any true face in that image: skip if >0.2 to avoid edge of a face.
16. Use NMS with a high overlap threshold (like 0.1 or none at all) just to reduce duplicates.

17. Sort these candidates by score descending and take top ~N. UT suggestion was often ~hard negatives equal to the number of initial negatives [24] [25]. If we had 10000 initial neg, maybe add 10000. But that might be overkill unless we see many FPs. Perhaps add a few thousand. We can also see diminishing returns: the highest scoring FPs are most valuable.
18. Add them to our `X_neg` and `y_neg` (which is 0).
19. Retrain SVM:
    - We can either retrain from scratch with all data (slower but straightforward) or do incremental update. Given not too big data, retrain from scratch with LinearSVC again. Or even use the previous `w` as initialization if we implement our own SGD, but not needed.
    - Possibly adjust C if needed (maybe with more data, one might reduce C a bit to avoid overfitting).
20. After retraining, evaluate on validation again:
    - Expect to see some specific false positives gone (particularly those we mined).
    - Possibly overall precision at high scores improves (AP may increase a few points).
    - Check that recall didn't drop drastically. If it did, maybe we added something that conflicts with faces (like half face).
21. Save the new model (just in case, e.g. `clf_hnm`).

22. We could do a second iteration: If new model still has some false positives (maybe different ones now), you could mine again. But time might be limited; decide based on AP improvement if another round is worthwhile. Usually one round is fine.

23. **Tuning and Finalizing Detector:** With the final model:

24. Possibly re-run it on validation to get final PR curve (that will be our best detector's performance).
25. We can now choose an operating threshold if needed for e.g. the bonus or final demonstration. For instance, if we want to output final bounding boxes for demonstration images, we might choose a threshold that yields a decent precision (~say precision 0.5).
26. But since the evaluation likely uses AP, just output all detections with scores.
27. Summarize improvements: "After HNM, AP increased from X to Y, precision at 0.5 recall improved, etc."
28. Save new PR curve plot `debug/pr_curve_final.png` (or overlay with initial to show difference).
29. Also gather some qualitative examples:
    - **Good cases:** where initial missed a face but final detects it (e.g. a face that was large or at a tricky scale).
    - **Failure cases:** Are there still certain faces not detected? Maybe extreme pose or very small ones. Document them.
    - **False positives examples:** Are there still some? Perhaps yes – list top 3 remaining false positives and maybe manually see what they are (maybe some other character drawing on wall that looks like a face, etc.). At some point, with a linear classifier, you can't eliminate all without risking missing some true faces, so there's a balance.

Common Pitfalls (M4) and Fixes:
- *Symptom:* **Detector becomes overfit after HNM (recall drops).**
Cause: Possibly introduced a negative that was actually part of a face or very close to a face boundary, causing the classifier to also suppress some genuine face features. Or simply too many hard negatives skewed it.
Diagnosis: Compare detection output before vs after HNM for true faces. If some faces had lower scores or

got missed, likely related to those added negatives.
Fix: Refine negative selection: exclude any borderline cases (maybe increase IoU threshold to 0.1 or 0.2 when filtering, to avoid including even patches that partly overlap a face). Also maybe limit the number of hard negatives so as not to overwhelm positives. You could also decrease C (to not force classifier to fit them too hard, thereby keeping a bit of slack). Try a smaller C (which means more regularization, so it won't penalize missing those hard negatives as heavily, potentially preserving some recall). This is tricky – ideally the selection was careful enough. Another approach: weight the hard negatives less in training (not straightforward in SVM without adjusting C per sample, though class_weight could be used but that's global for all neg vs pos).

- *Symptom:* **No significant improvement after HNM.**
  Cause: Possibly the initial classifier already had enough negatives or the false positives were mainly due to scale issues rather than classifier confusion. Or our threshold for adding was too high (didn't add many negatives).
  Diagnosis: Check how many and what kind of patches were added. If only few or very similar ones, maybe they didn't generalize.
  Fix: Add more negatives or ones with slightly lower scores too. Also ensure diversity: if all hard negs came from one image region, maybe not enough variety. Possibly gather from more images or lower the threshold to get different negatives.

- Also verify if AP changed: sometimes AP might not jump if the few removed false positives were at lower score ranks or replaced by other FPs. But a cleaner precision at top ranks might still be achieved. *Note:* It's possible a linear classifier saturates and more data yields diminishing returns – the cited student result even showed a slight drop after HNM [25], hypothesizing the extra negatives weren't representative of test distribution. We should monitor if AP oddly decreases, and if so, we might revert or try a different approach (e.g., not include those or try an RBF kernel, but that's out of scope).

- *Symptom:* **Detector still misses some faces (especially smaller ones).**
  Cause: Perhaps faces smaller than 36 were present and we didn't upsample or our pyramid didn't go far enough.
  Diagnosis: Look at missed faces and their size in pixels. If e.g. a 20×20 face is always missed, that's because our detector can't handle that size (36 base).
  Fix: If required, one could train another detector with smaller window or try upscaling images. But given project constraints, likely we accept a minimum face size limit. We can mention this limitation in the report.

- *Symptom:* **Runtime too slow due to multi-scale.**
  Cause: Many scales + large images + fine step = lots of windows (e.g., a 800×600 image has ~ (800/6 * 600/6) ~ (133*100)=13k windows per scale; at 10 scales that's 130k evaluations – borderline but maybe okay if using optimized C code inside sklearn for decision_function. If it's pure Python per window, that's slow. But we can vectorize by computing HOG map and convolving with the SVM filter which is what DalalTriggs did).*
  Diagnosis: *If detection on one image takes multiple seconds, that could be an issue if test set is large. But if it's manageable (say <1s per image) it's fine.*
  Fix:* Optimize detection: possibly precompute hog for full image and then slide (which is effectively a convolution of the HOG feature map with the HOG template). There are known techniques (the

classifier weight vector can be reshaped into the same 5x5x36 block structure and you can do cross-correlation). If needed and time permits, implement an optimized sliding window:

    - Compute full HOG once, then use convolution (e.g., use `signal.correlate` with the w reshaped as filter on the HOG channels).
    - Or at least avoid computing hog repeatedly: maintain the HOG map and use array slicing as mentioned. This is advanced, possibly not required by the assignment if performance is okay.
- If not optimizing, maybe just mention as future improvement in report.

By the end of M4, we should have a robust face detector. This completes Task 1. The expectation is that the AP is reasonably high (depending on dataset difficulty). We should now be outputting the final `detections_all_faces.npy` and associated files for the test set with this model. Next, we leverage this to do Task 2 (character recognition).

8. Milestone M5: Character Recognition Extension (Task 2)

Concepts: Now that we can detect faces, we add a classification step to identify which character the face belongs to (Daphne, Fred, Shaggy, Velma). This becomes a **multi-class classification** problem once a face is detected. We will either train separate classifiers or a single multi-class model. We also evaluate performance in terms of confusion matrix and per-class accuracy.

- **Approach 1: One-vs-All Detectors:** Train 4 separate detectors, one for each character. Essentially, treat each character's faces as positive and all other faces (and possibly backgrounds) as negatives for that classifier. Then run all four detectors on an image. This is like having four SVMs, each will fire on any face but ideally strongest on its target class. Then we label a detection by whichever SVM gave the highest score (if any are above threshold). This approach might yield multiple detections (one face triggering multiple classifiers) so some coordination (like a softmax or picking max) is needed. This is somewhat complex and also not how the output format is structured (they want separate lists per class).
- **Approach 2: Two-Stage Classification:** Simpler: Use the face detector (binary) to find face regions, then classify those regions using a multi-class classifier. That classifier can be a linear SVM (multiclass) or even a decision based on 4 binary SVMs (like approach1 but applied only to extracted face patch, not scanning).
- Essentially, once a face is found at location, we crop it (or use the HOG we already computed for it) and classify into 4 classes. This way we only deal with one face patch at a time in classification, not scanning entire image separately for each class.
- The assumption is that face detection already handled finding the faces (which are common for all classes), now it's just labeling.
- We need training data for this classifier: we have all training face patches labeled by character from the annotations. That we prepared in M2.
- We might use the same HOG features but now label them by class (0-3 or by name). Or we could explore other features (color might help differentiate e.g. Daphne with orange hair vs Velma with brown hair). But given HOG and grayscale approach, maybe the classifier might focus on outline differences (Velma has glasses, Shaggy has a goatee, etc – HOG might capture some of that).
- Possibly using color could improve – if allowed, one could append color histograms to feature vector or use a simple color-based rule as supplement. But to keep scope defined, we stick to HOG.

- **Train multi-class SVM:** Sklearn's LinearSVC by default does one-vs-rest for multi-class if y has more than two unique values. So we can feed it X = HOG features of all faces, y = class index (0-3). It will train 4 linear classifiers internally. We should probably use a smaller C if classes overlap a lot, or try default.
- Alternatively, train 4 separate SVMs ourselves: e.g., for Daphne vs not-Daphne, etc. But since we ultimately need a single predicted label, one-vs-rest with decision_function outputs for each class works. It gives an array of shape (n_samples, 4). The highest score class is predicted.
- Check training accuracy to ensure it can separate them to some degree.
- **Integrating detection and recognition:**
- Option A: Run face detector on an image, get boxes. For each box (face), crop and compute HOG, then apply the multi-class classifier to label it. That's straightforward and ensures we only label things we are fairly sure are faces.
- Option B: Do it in one step: we could have a unified detector that directly classifies into 5 classes (4 characters + background). That would involve training a multi-class detector on window features. We could have done that by labeling positives by class and labeling negatives as "background" class. LinearSVC multi-class would then give 5 decision values. The advantage is you get detection and classification in one pass. The disadvantage is potentially more confusion between classes and background if not handled carefully. Given the time, sticking to two-stage is safer.
- The output format requiring separate .npy per class suggests they likely expect the final step to produce per-class detections. If we do two-stage, we can easily populate those arrays by filtering the detected faces by predicted class.
- We need to ensure each face detection yields exactly one class label prediction. If the classifier is unsure or if the face detector gave a false positive (which isn't actually a face), the classifier will still label it as one of the four classes (since multi-class SVM will always choose one). That could result in a false positive being counted as a false detection for some class. But if our face detector is good, most should be actual faces.
- If a face is missed by detector, it's missed entirely (FN for that class, no detection output).
- If a face is detected but misclassified, then it's a FN for the true class and FP for the predicted class. The confusion matrix will capture that.
- **Evaluation (Task 2):** Likely they will consider a detection correct if it is correctly localized *and* the correct class. They gave GT for each class, probably meaning:
- They might evaluate detection per class as separate (like AP for each class or recall for each class).
- Or they build confusion as: for each ground truth face of class X, if a detection overlaps it:
    - if that detection is labeled X, it's a TP for X.
    - if labeled Y (wrong), that's a misclassification (count as FN for X, FP for Y).
- They might then compute precision, recall per class, maybe AP per class as well if sorting by scores class-wise.
- The confusion matrix is a good summary to include in the report, and class precision/recall or F1 can be derived from it.
- We will produce the arrays `detections_<class>.npy` etc. Each should contain the boxes of that class. We likely already had global detection list; we can partition it by predicted label. But careful: If two classes might have given detection on same face from earlier multi-detector approach, in our approach it won't happen because we pick one label per detection. So each detection goes to exactly one class's list.
- We should maintain separate score arrays per class as well. We can reuse the detection confidence from face detector or combine with classification confidence. Usually, a final confidence for class might be the face score *and* classification confidence. But since classification is separate, its output is

not a calibrated probability. A simple way: the face detector's score is how confident it's a face; the classification decision just assigns a label. For ranking within one class's detections, maybe multiply the face score by some factor if classification was not certain – but LinearSVC one-vs-rest scores can be used: e.g., we have decision values for all 4 classes, the highest one is chosen, but its magnitude could indicate confidence for that class.

- Perhaps simplest: We keep the face detection score as the score for the detection in all classes, or we could use the one-vs-rest score of the predicted class as the score. The evaluation might not need fine ranking per class beyond counting correct matches (if they do AP per class, we should provide a score).
- Possibly best: For each detection we output into class X, use the SVM decision value for class X as score. That might better reflect confidence that it's specifically that class (if one-vs-rest, decision_function for each class vs rest). This way if a detection could also somewhat look like class Y, its score for X might not be huge.
- We'll do: `scores_class[i] = multiSVM.decision_function(face_feat)[predicted_class]`. Sklearn's LinearSVC in multi will give an array of shape (n_class,) for decision function (for OvR, these can be interpreted as signed distance to each class hyperplane).
- Make sure to use the right values: If using OvR, there is a separate hyperplane for each class vs all others. So the score for the predicted class hyperplane can serve as confidence. (It could be negative if the sample is below margin but still highest among classes, but that would mean classifier is not very confident any class – if that happens often, maybe classes overlap strongly).
- It might be simpler to just output face detector's score as a base confidence and assume classification is mostly correct or not – but for AP per class, a misclassified detection counts as an FP for that class with some score – using face score is fine in that scenario because it was how confident it's a face (it was a face, just wrong identity).
- This detail might not significantly alter evaluation, so either is acceptable. We can mention we use face score or combined.

Implementation Steps (M5):

1. **Prepare multi-class training data:** Using the same positive face patches from M2:
2. Create label array where `label[i] ∈ {0,1,2,3}` corresponding to ["daphne", "fred", "shaggy", "velma"] (assuming these four).
3. We likely have them separated originally by reading each annotation file. So it's straightforward: e.g., all face patches from Daphne's annotation get label 0, etc.
4. Ensure roughly balanced: If one class has very few samples (maybe not, but check counts). If very imbalanced, consider `class_weight='balanced'` in SVC.
5. Train a LinearSVC with `multi_class='ovr'` (default).
6. E.g., `clf_char = LinearSVC(C=0.01, random_state=0)`.
7. Fit on HOG features of all training faces.
8. Check training accuracy (expected quite high if faces are distinctive, but if they look similar maybe moderate). If low, might need to increase model complexity or add features (not in scope to change to non-linear, but one could try polynomial kernel or RBF – probably overkill).
9. Possibly do a quick cross-validate by holding out a few training images to ensure it generalizes (not always needed in these projects, but mention if time).
10. If performance is suboptimal, maybe the HOG features lose color/clothing info. Could consider concatenating a color histogram (like 16-bin hue histogram). Since Scooby-Doo characters have

distinct color schemes, that could boost it. But if not allowed external libs, we could manually compute color hist and attach. If we suspect needed, mention as potential improvement.

11. **Detect & Classify on Validation:** Using the final face detector (from M4):

12. Run it on each validation image to get face boxes (with high recall hopefully).
13. For each detected face box, crop the image region (the original image) corresponding to that box (we might have to pad or ensure same size as training patch? Since our classifier expects 36x36 input as well, so likely we should resize the detected face region to 36×36 just as we did for training).
14. Compute HOG feature for that patch (or if the detection pipeline can carry the HOG feature it used, we could re-use it if same scale… but easier to recompute to avoid error).
15. Pass that feature to `clf_char.predict()` to get a label.
16. Also get decision_function scores for that sample.
17. Assign the detection to that class's list with the same coordinates.
18. Score: as discussed, perhaps use the face detection score multiplied by some factor or the char classifier score. For simplicity, we can use face score as base because it already measures confidence of a face being there (if classifier mislabels a face, it's still a face, just wrong identity, so in class evaluation it counts as FP anyway). Alternatively, multiply face_score by a confidence factor for the class (for instance, if the char classifier's margin for the chosen class is low, maybe downweight it).
19. However, since evaluation likely per class AP, we want high ranking for truly correct ones. Using face score alone might rank false-labeled faces high too because the face score was high even if the identity was wrong.
20. Possibly better: use `score_class = face_score * (decision_value_of_predicted_class)` as combined confidence. If the classifier isn't confident (decision val small), this product lowers the score. If classifier is very confident (large margin) and face was confidently detected, product is high.
21. But this is getting fancy. The spec doesn't specify how to rank; any reasonable scheme is fine.

22. We'll assume using face score for detection ranking is acceptable and mention that more refined scoring could incorporate classification confidence.

23. After processing all images, we have `detections_daphne, scores_daphne, file_names_daphne` etc.

24. Save them to `.npy`. Ensure to match exactly the names expected (the structure given in user's folder listing shows e.g. `scores_velma.npy` etc). Check if they likely want all-lowercase names matching those files (yes they listed in structure all lowercase class names).

25. These arrays lengths will vary by class depending on how many detections of each class our system made.

26. **Evaluate Task 2 on Validation:**

27. Using evaluation script if available (it may produce confusion or per-class precision).
28. Or manually:
    - We know each GT face has a class. For each GT face, see if our outputs have a detection overlapping it (≥0.5 IoU) in the correct class output. If yes, that's a True Positive for that class.

If we have a detection on it but in wrong class output, that's a miss for the true class and a false positive for the wrong class.
    - Some detection might not match any GT (false positive for whatever class we labeled).
    - Compute confusion matrix: create a 4x4 matrix where cell (i,j) counts faces of class i that were predicted as class j. You get these by matching each GT: if matched by a detection, increment (true_class, predicted_class). If not matched at all, that's a miss – could count as (true_class, none). But in confusion matrix we usually ignore the "none" (we could consider an extra column for "missed"). But since our system either outputs something or not, misses are just not tallied in any predicted class, so you can derive them as remaining GT minus hits. For completeness, we might make it 5x5 with a "None" category for no detection, but not necessary for report.
    - Also count total detections per class vs how many of them matched (precision by class = TP_pred_class / total_pred_class; recall by class = TP_true_class / total_true_class).
    - Build the confusion matrix image: show percentage or count in each cell. The diagonal ideally high numbers, off-diagonals lower.
29. Typically, we present Precision, Recall for each class and maybe mean or overall accuracy (which is sum of diagonal / total faces).
30. If one class has much lower metrics, analyze why (maybe fewer training examples or the detector tends to miss them due to differences in appearance or position in images).
31. Example: maybe Velma (with glasses) vs Daphne (no glasses) might confuse if both have similar hairstyles shapes in HOG. Or Shaggy vs Fred (both male faces but Fred has a different hair shape). See if confusion matrix shows specific confusions (like many Velmas predicted as Daphnes, etc.), and note that.

32. These insights can suggest improvements: e.g., add color info because Daphne's hair is orange vs Velma's brown; HOG grayscale can't tell that.

33. **Visualization:**

34. Perhaps create a few images showing final result: draw bounding box and put the **name** label on each detected face in an image. Use different colors for different predicted classes to make it clear. Save as `debug/detections_with_labels.png`.
35. Also could illustrate a failure: e.g., an image where it mislabeled someone – draw what it predicted vs actual (maybe in caption rather than on image).
36. The confusion matrix figure is important to include in report.
37. Also a bar chart of per-class precision/recall might be nice or a small table, but the confusion matrix covers it.

Common Pitfalls (M5) and Fixes:
- *Symptom:* **Character classifier has low accuracy.**
Cause: Features not discriminative enough or classes too similar. Possibly our HOG on faces (especially cartoon where faces might be similarly shaped) might not separate well.
Diagnosis: Check training accuracy. If even training is low, model underfit – try C up, or features are insufficient. If training is high but validation poor, overfit – maybe not enough data.
Fix: Add features: e.g., include color or pattern features. Scooby-Doo characters have distinguishing colors (Velma's orange sweater, Daphne's purple dress, etc.). One simple way: when computing HOG, we used grayscale. Perhaps try using color HOG (skimage.hog on each channel and concatenate – might help catch color edges differences). Or append a color histogram of the face patch (e.g., 16 bins of hue) to the feature

vector. If allowed, that could boost it. If not adding features, maybe use a slightly lower regularization (C) to generalize better, especially if some class had few examples (class imbalance). We could also do data augmentation: horizontally flip face images to double data (assuming a flipped face is still same character – yes, presumably). That could help if limited data. Mention these if relevant.

- *Symptom:* **Mislabeled detections (confusions) remain high between certain classes.**
  Cause: Some characters look alike in grayscale edge terms or appear in similar contexts. E.g., Daphne vs Velma might confuse if glasses not strongly captured.
  Diagnosis: Look at misclassified examples. Are there patterns? Maybe the angle of face or partial occlusion? Maybe our face detection window sometimes includes some clothing which confuses (like Velma's orange sweater vs Daphne's purple one – grayscale HOG sees similar intensity maybe).
  Fix: If time, consider training the classifier on not just the face but a slightly larger patch including hair/outfit. But that's a new approach. Or accept that at least detection was correct, classification is secondary. Possibly calibrate thresholds: e.g., if a certain class tends to get over-predicted, maybe raise its threshold or apply a prior (if we know in dataset maybe all classes appear equally often or not). *Note:* In report, one can acknowledge such confusions and suggest possible fixes (color, more data, ensemble of classifiers, etc.). It's part of learning to analyze errors.

- *Symptom:* **Output format issues for Task 2.**
  Cause: Possibly mixing up which detection goes to which class .npy, or misalignment of file_names.
  Diagnosis: Double-check that the number of entries in `detections_class.npy` equals that in `scores_class.npy` equals that in `file_names_class.npy`. If not, there's a bug in how we collected them.
  Fix: Ensure when appending detection and score to a class list, also append the corresponding filename string. Sort each class's detections by score if needed (not sure if required, but likely yes to compute AP per class if they do). We can sort within each class list by descending score just to be safe.

- *Symptom:* **A detected face gets output in the wrong class file (because misclassified).**
  Cause: That's intended: it's a misclassification. But just ensure that face's coordinates appear only in one class's output. If our logic accidentally allowed the same box to go into multiple class outputs (shouldn't happen with two-stage, but if one tried one-stage with 4 detectors, that could happen).
  Diagnosis/Fix: With two-stage method, one detection = one class label, so fine.

Now the system is complete: it detects faces and labels them. We would then produce final outputs on the `testare` set (for which we don't have GT, but we would run the detector & classifier and save task1 and task2 .npy files). That's what would be submitted for evaluation. We should test our pipeline on validation to ensure everything is smooth before running on test.

9. Reproducibility and Testability

To ensure the project is **reliable, reproducible, and easy to test/debug**, we adopt several best practices:

- **Centralize Parameters:** All tunable parameters (window size, HOG cell size, SVM C value, pyramid scale, step size, NMS IoU threshold, etc.) are defined in one place (e.g., `Parameters.py` or a config section at top of `RunProject.py`). This makes it easy to adjust and to document what values were

used. For example, `Parameters.dim_window = 36`, `Parameters.scale_factor = 0.9`, `Parameters.threshold = 0.0`, etc. The code then references these, so changing a value in Parameters immediately applies everywhere. This also helps the TA/teacher quickly see what settings were used by reading one file [7] [26].

- **Set Random Seeds:** Wherever randomness is involved, set the seed to fixed value for consistency. For instance, in negative sampling, use `random.seed(123)` before selecting random crops. In training SVM, specify `random_state`. This ensures that each run of the pipeline yields the same results (makes debugging easier and the evaluation consistent).
- **Cache Intermediate Results:** Some steps are time-consuming, like computing HOG features or training SVM. Save these results so you can reuse them without recomputation:
- Save the feature matrices (`pos_features.npy`, `neg_features.npy`) after M2. Then in M3, you can load them directly if they already exist. This way, if you tweak SVM parameters, you don't need to recompute HOG from scratch.
- Similarly, after training a model, you could serialize it (pickle the `clf` or save `w,b` to a file) so that you can quickly reload and run detection without retraining, if needed.
- Logging which cache files correspond to which parameters (maybe include a hash or just be careful to clear caches when parameters change).
- **Logging and Hyperparameter Tracking:** Print or save to a log file the key hyperparameters and results of each experiment run. For example:
- When training SVM: output the value of C, number of support vectors or iterations, training accuracy.
- When running detection: output how many detections were produced in each image, etc.
- When evaluating: print AP, per-class accuracy, etc. in a summary.
- If doing multiple trials (like trying different C or cell sizes), keep notes or a results.txt with those outcomes to compare.
- **Unit Tests for Core Functions:** Implement small tests for functions like `compute_iou`, `non_max_suppression`, `hog_extraction`:
- We already discussed IoU testing with known boxes. We can include that in code (perhaps under a `if __name__ == "__main__":` block or a separate test script).
- Test NMS: e.g., feed two boxes with IoU > 0.3 and see if it removes one. Also test two far-apart boxes remain. Possibly manually craft:

```
test_boxes = [[0,0,10,10, 0.9], [1,1,11,11, 0.8], [100,100,110,110, 0.7]]
# last number could be score
# First two overlap heavily (IoU ~0.81). NMS should keep the 0.9 and drop
the 0.8, and keep the third.
kept = non_max_suppression(test_boxes, iou_thresh=0.5)
assert len(kept) == 2 and [0,0,10,10] in kept and [100,100,110,110] in kept
print("[OK] NMS suppresses overlapping boxes correctly")
```

(This requires adapting NMS to the test format; it's conceptual.)

- Test sliding window logic on a tiny image: e.g., create a small synthetic image (like 36x36 with a simple pattern), run detection and ensure at least at (0,0) one window is considered. This is tricky without a proper model, but we can at least ensure our loops cover the image edges properly (no off-by-one errors).
- Test that our detection output arrays align: e.g., after creating them, assert `len(detections)==len(scores)==len(file_names)`.

- **Modularize Code for Testing:** Structure the project into functions that can be called independently:
- e.g., `extract_hog_features(image_list)` returns feature arrays (easy to test by giving it one known image).
- `train_svm(X,y)` returns a trained model (we can test it quickly on a trivial dataset to see if it outputs expected coefficients).
- `detect_faces(image, model)` returns detections for one image (we can unit test on a fake model: like use a dummy `model` that always returns positive score to see if the function returns all windows, etc).
- This makes it easier to debug each part in isolation.
- **Use Debug Flags and Intermediate Outputs:** Have a mode or flag (like `debug=True`) where the code produces extra prints or images. For example, during development, you might output the HOG visualization for every 10th face to ensure they're okay. Or print `[OK] Processed image X, found Y raw detections, Z after NMS`.
- Once the code is stable, you can turn off verbose debug to speed it up, but it's very useful to have those while building. Also consider assertions in code to catch anomalies (e.g., `assert hog_vec.size == expected_len` after computing a HOG).
- **Consistency in Training vs Testing:** Make sure any preprocessing you do on training data is identically done on test data. For instance, if you resized training faces to 36×36 and converted to grayscale before HOG, do exactly the same for detected faces before classification. Keep functions for these transformations to reuse. If using OpenCV for one and PIL for another, slight differences could affect results – avoid that by a uniform pipeline.
- **Experiment Management:** If exploring different parameter values (say cell size 4 vs 6, or adding hard negatives vs not), consider structuring code to allow easy switching and maybe produce separate output files (so you don't overwrite a working solution's outputs). Possibly use a config version or command-line arguments to run variations. This can also be part of reproducibility (someone should be able to reproduce the final results by setting the parameters as documented).
- **Random Seed for Hard Negative Mining selection:** If picking top N by score, that's deterministic. If picking random negatives initial, we seeded. Ensure any randomness in those steps is controlled so that retraining with or without HNM is comparable and not subject to random fluctuations.

Adhering to these practices ensures that if something goes wrong or results seem off, we can systematically isolate the cause. It also means someone else (e.g., the instructor) can run our code on their machine and get the same outputs and verify the logic with confidence.

10. Starting from a Minimal Baseline & Incremental Improvements

We followed a strategy of **starting with a simple baseline and then incrementally adding improvements**, which is crucial for such projects:

1. **M0 Baseline (No detector):** Begin with a skeleton that outputs no detections or random detections – essentially verifying the pipeline format. This ensures the evaluation system is satisfied with the output file formats even before we have a working model. (In code, we had the model predict random faces if unmodified [6], which gave an initial PR curve near chance [27].)

2. **M1/M2 Basic Data & Features:** Next, implement the core feature extraction (HOG) and use a single-scale sliding window with a linear classifier. This gave us a **minimal face detector** that works only at one size. It's not great (perhaps ~0.2–0.3 AP) but it's a functioning detector [2]. This baseline is

important as a reference and a starting point to debug (we confirmed it detects some faces, and we got an initial PR plot).

3. **Improvement 1 – Multi-Scale Detection:** We then integrated an image pyramid to catch faces of different sizes. This boosted recall significantly (our expectation: AP jumps closer to e.g. 0.8 if done well [21], as multi-scale is critical). We saw more faces detected that were missed at baseline.

4. **Improvement 2 – Non-Maximum Suppression:** This is actually implemented alongside detection. It doesn't necessarily change AP (since duplicates don't increase TP count and might increase FP count if left in), but it makes the detector output cleaner (one detection per face). It slightly improves precision by removing multiple detections on the same face. It's standard in any sliding-window detector [19].

5. **Improvement 3 – Hard Negative Mining:** We ran the detector on difficult backgrounds and added false alarms to training. This typically yields a modest increase in precision (5–10% improvement reported [22], depending on how confusable the background is). For us, maybe certain background patterns in cartoons triggered false faces, which HNM addressed. The result is fewer false positives at the same detection threshold, thus a higher precision for recall levels, lifting AP further.

6. **Optional Tuning:** We could further fine-tune things like the sliding step (smaller step to catch faces between cells might improve AP at cost of runtime [26]) or tweak the SVM threshold for final output if we wanted a particular precision-recall operating point.

7. **Improvement 4 – Character Classification:** Finally, building on the robust face detector, we add the recognition layer. This doesn't feed back into the detector (so it doesn't change Task 1 performance), but it completes the system for Task 2. We train on the baseline of detected faces (which themselves are result of improvements 1–3) to assign labels. We likely started with a simple multi-class linear classifier. If we had more time or needed improvements:

8. Could incorporate color or train more advanced classifier if needed.
9. Could use an ensemble or one-vs-one classifiers, but probably not needed as linear is sufficient given distinct appearances.

Each step was validated before moving on: e.g., we ensured the single-scale detector worked before tackling multi-scale, ensured multi-scale was working (by visualizing detections at various scales) before doing HNM, etc. This incremental approach helps isolate bugs and also see the contribution of each improvement: - We could plot the PR curves of baseline vs multi-scale vs multi-scale+HNM to show progressive improvement. - Also time complexity increase: baseline was fastest, multi-scale slower, HNM adds training time but detection time unchanged, classification adds a tiny overhead per face.

We prioritized improvements that yielded the most gain first (multi-scale), then those that fine-tune (HNM). We avoided premature optimization (like we didn't immediately try to optimize feature computation until we saw if runtime was an issue).

By the end, the **final detector** (multi-scale + HNM) combined with the classifier yields strong results: likely high recall of faces and decent accuracy in character recognition. Any further improvements (like using a cascade or deep learning) are outside scope, but could be mentioned as future work.

11. Final Submission Checklist

Before considering the project complete, go through this checklist to ensure all deliverables are ready and the report is complete:

- **Code & Structure:**
- All source code files are present in the `cod/` directory, with correct names and no extraneous files. Verify that running `RunProject.py` (or the main script) goes through the full pipeline and produces the expected output files without manual intervention.
- Remove or disable any debug-only code (excess prints or slow visualizations) for the final run, or guard them under a debug flag so they don't clutter output or slow down when not needed.
- Ensure any paths in code are relative or conform to the structure so the TA can run it. (E.g., no absolute paths to your local machine; use `os.path.join` with base directories from Parameters).

- Double-check that `evalueaza_solutie.py` runs on your output folder without errors. If possible, run it one more time on the final outputs.

- **Output Files (.npy):**

- **Task 1:** Three files – `detections_all_faces.npy`, `scores_all_faces.npy`, `file_names_all_faces.npy` – should be in the designated solution folder for task1. Verify their shapes:
  - `detections_all_faces.npy`: shape (N,4) where N is total detections, each row [x_min, y_min, x_max, y_max] (integers).
  - `scores_all_faces.npy`: shape (N,) floats corresponding to each detection's confidence.
  - `file_names_all_faces.npy`: shape (N,) with each entry a string (filename) corresponding to the detection. (Make sure the filenames match exactly the test image names expected – e.g., including extension).
- **Task 2:** Nine files (if 4 classes):
  - `detections_daphne.npy`, `scores_daphne.npy`, `file_names_daphne.npy` (and similarly for fred, shaggy, velma).
  - Check each triple: lengths match and sum of lengths of all class detections = N (total faces detected).
  - Filenames should appear in the class file that indeed contain that class's face (if a detection was misclassified, it will appear under the wrong class – that's expected).
  - It's worth scanning a few entries of each to see if any obvious errors (e.g., a file name appearing under two classes might mean duplicate detection; ideally one detection one class).

- All .npy should be saved with `allow_pickle=False` (default for numpy save) to avoid any security issue – likely fine.

- **Figures for Report:** Prepare a set of key figures to include in the PDF (make sure to save them at good resolution):

- **Precision-Recall curve (Task 1):** Show the performance of your face detector. Ideally, include at least the final PR curve. Optionally, overlay baseline vs final to show improvement. Label axes (Precision, Recall) and mark Average Precision. For example, "AP = 0.85" on the graph [20].
- **Confusion Matrix (Task 2):** A 4x4 matrix image with actual vs predicted classes. You can use `sklearn.metrics.confusion_matrix` and plot it with `plt.imshow` plus annotations. Include class labels on x and y axes. Possibly normalize to percentages to highlight errors. This will directly address how well character recognition did.
- **Example Detection Images:** At least one figure with multiple sub-images:
    - One showing detections on an image (with boxes drawn).
    - Possibly one illustrating NMS effect (before/after) on a zoomed-in region.
    - One per class if possible (like an image where each of the four characters is detected and labeled correctly).
    - If any interesting failure, maybe show that too (e.g., "the detector thought Scooby's face on a poster was a human face of Shaggy" or something fun).
    - Make sure any images in report are clear and not too small.
- **HOG Visualization (optional):** Perhaps include a small figure of the learned HOG template for "face vs non-face". This often looks like a face shape in the gradients. It's a nice visual to show what the classifier has learned. (It was seen in Dalal-Triggs and assignments often include it) [28]. You can create this by taking the SVM weight vector and reshaping to the HOG cell grid (5x5x36) and plotting as an image of gradients.
- **Hard Negative Mining effect:** If you plotted PR before and after, include that or a small bar chart of AP improvement.

- **Any other diagnostics:** e.g., the histogram of face sizes (to show why multi-scale was needed). Possibly relegated to appendix if space, but could be used to justify design choices.

- **Tables or Metrics:** In the text, you should quote numbers:

- Number of training images, number of positive/negative samples used.
- Final average precision on validation for detection.
- Precision and recall for each class in recognition, and overall accuracy. (A small table could be: Class, Precision, Recall, F1).
- Perhaps runtime info: how long to train, how long to run on, say, 10 images (to show it's efficient enough).

- If relevant, list parameter settings (maybe in an appendix or footnote in report). Or just ensure they're mentioned in text ("We used window size 36, HOG cell 6, block 2x2, SVM C=0.01, NMS threshold 0.3, pyramid scale 0.9, step 6px").

- **Report Write-up (2+ pages):** The report should be concise but contain:

- **Introduction:** State the problem (face detection and recognition in Scooby-Doo images) and the classical approach (sliding window, HOG, SVM).
- **Methodology:** Describe your implemented pipeline. You can follow the milestones as sections or integrate:
    - Data preparation, HOG features,
    - Classifier training,
    - Sliding-window detection and NMS,

    - Hard negative mining,
    - Multi-class classification. Emphasize decisions you made (like “we chose cell size 6 px after trying 4 and 8 because it gave a good balance”, or “we did one round of hard negatives which improved precision by X”).
- **Results:** Present the PR curve and AP for face detection. Compare it to any baseline if applicable. Mention how it meets expectations (e.g., “Our detector achieves AP=0.85 on validation, which is high given the task. The precision at 90% recall is around 80%, showing few false positives at high recall.”).
- For recognition, present confusion matrix and discuss which classes are easiest vs hardest. Provide overall accuracy (e.g., “Overall, character recognition accuracy is 95% on validation; only 2 Velma faces were mistaken for Daphne”).
- Perhaps include some discussion of failure cases (maybe one class is consistently confused or one face missed due to size – mention that).
- **Conclusion:** Summarize what was achieved and any possible improvements or future work. E.g., “With more time, we could try a cascade like Viola-Jones for speed [29], or incorporate color to boost recognition. One could also expand to detect the Scooby-Doo dog if needed, etc.” Show you understand what could be done beyond.
- Ensure citations if referring to known methods/papers (if required by assignment). Perhaps cite Dalal & Triggs (2005) for HOG or the course notes [9].

- Keep it to the point given page limit ~2-4 pages. Focus on key points and figures.

- **Final Code Run on Test Set:** Before zipping up, run your final code on the actual test images (if provided) to produce the output files to submit. These go into the folder structure exactly as required (likely `evaluare/fisiere_solutie/<YourID_Name>/task1/...` and `task2/...`). The eval script might be run by the TA to grade both tasks. Confirm that the number of detections seems reasonable (not zero or ridiculously many). Since we don’t have GT for test, just trust the model.

- Double-check that the code does not accidentally use validation or GT info when running on test (it shouldn’t, if structured properly).

- **Package and Submit:**

- Include the README if needed (with instructions how to run).
- The PDF report.
- Code in `cod/` and possibly `debug/` images (though they may not require debug images, but report has them).
- Not the dataset or any large intermediate files.

Going through this checklist will help catch any last-minute issues. Once everything is verified, you can submit with confidence that the system will run and the results in the report are backed by the outputs. The student will also have learned a lot through this iterative, rigorous process – understanding not just how to build a face detector, but how each component contributes and how to systematically improve a computer vision model using classical techniques. The final report will document this journey and the achieved performance, demonstrating a successful project execution.

---

1 9 10 14 16 Tema 3.pdf
file://file_000000006098720a99cdcebd8585ca62

2 4 6 20 23 26 29 Assignment 5: Face detection
https://vision.cs.utexas.edu/376-spring2018/assignments/a5/A5.html

3 5 22 cs143 / face detection
https://cs.brown.edu/courses/cs143/2011/results/proj4/psastras/

7 8 15 Parameters.py
file://file_00000000ff1471f4b8840752833da8c4

11 12 13 Intersection over Union (IoU) Explained | Ultralytics
https://www.ultralytics.com/glossary/intersection-over-union-iou

17 18 Intersection over Union (IoU): Definition, Calculation, Code
https://www.v7labs.com/blog/intersection-over-union-guide

19 image processing - non-maximum suppression on detection windows - Stack Overflow
https://stackoverflow.com/questions/15785350/non-maximum-suppression-on-detection-windows

21 24 25 27 28 Face Detection Project
https://jlin.xyz/compvision/proj5/html/index.html